# DES Engine
## Reference Manual

A discrete-event simulation engine in the spirit of Arena's Basic Process template, written from the simulation theory table up. This manual lists every public type and member, says what each one is for, and traces what actually calls what when a model runs.

The signatures here are **extracted from the headers** when the manual is built, so they cannot drift from the code. Only the prose is written by hand.

*v16 -- the ride-hailing model - 1630 checks - 135 types*

## Contents

**1. Reading This Manual**

What is in here, how it was produced, and the shape of the engine as a whole.

**2. Foundations**

Time, entities, events, and the accumulators every reported number comes from.

**3. Queues and Resources**

Waiting lines, the disciplines that order them, and the servers they wait for.

**4. Randomness**

One primitive, twelve distributions, and the tests that show a generator is sound.

**5. The Expression Layer**

How a model's conditions, values and durations became text rather than compiled code.

**6. The Document Layer**

How a model stopped being a program and became a set of spreadsheets.

**7. Runtime Control**

How a run stopped being something you wait for and became something you drive.

**8. The Terminal UI**

How the spreadsheets got a screen, and how a terminal interface is tested.

**9. Editing a Model**

The text editor, the palette, and why the file itself is what gets edited.

**10. The Flowchart**

The blocks entities move through, and the narrow window each one gets on the engine.

**11. The Model**

What is simulated, kept separate from the machinery that simulates it.

**12. Running a Simulation**

The engine, the stopping rules, and the trace that makes a run checkable by hand.

**13. Experiments**

Why one run is one sample, and what to do about it.

**14. Call Flows**

What calls what, in order, when a model is built and run.

**15. Free Functions**

The factories and helpers that are not members of anything.

Chapter 1

# Reading This Manual

What is in here, how it was produced, and the shape of the engine as a whole.

This manual documents every public type and member of the engine. It is generated: **the signatures are read out of include/*.hpp when the PDF is built**, so they cannot drift from the code. Only the prose is written by hand. If a member appears here it exists; if one is deleted from the code it disappears from the manual on the next build.

### Conventions

| Shown | Means |
|---|---|
| class | Has invariants to protect; members default to private. |
| struct | A bag of values, all public. |
| enum class | A closed set of named alternatives. |
| inherits | Base classes, so the polymorphism is visible. |
| I-prefix | An interface: pure virtual, implemented elsewhere. |

Members are listed in header order rather than alphabetically, because header order is how the author grouped them and that grouping carries meaning.

### The engine in layers

**Experiments**
`Experiment - Summary - replications, intervals`

**Engine**
`SimulationSystem - Clock - FutureEventList - Trace`

**Model**
`Model - INode - Station - Resource`

**Expressions**
`Parser - IExpression - EvalContext - VariableStore`

**Foundations**
`Entity - Statistics - RandomStream - Diagnostic`

*Each layer uses the ones below it. The expression layer sits at the bottom rather than beside the model, because it must not depend on Model: it declares IModelState and the engine implements it.*

### The one rule worth knowing first

Inside the run loop the integrals for the interval that just ended are closed **before** the clock moves, and the clock moves **before** any state changes. Swap any two of those and every time-weighted average goes quietly wrong, with no error anywhere. Chapter 13 shows the order explicitly.

Chapter 2

# Foundations

Time, entities, events, and the accumulators every reported number comes from.

## EventType

*enum class · Common.hpp*

What kind of thing an event notice represents. The engine dispatches on this in the run loop.

**Values** Arrival, StartService, Departure, EndSimulation, WarmUpEnd, Observe, Renege

## QueueDiscipline

*enum class · Common.hpp*

The service order a queue uses. Selects which IQueueRule a station is built with.

**Values** FIFO, LIFO, Priority, Random, SPT, EDD

## ResourceState

*enum class · Common.hpp*

Whether a resource is idle, partly busy, or fully seized.

**Values** Idle, Busy

## Clock

*class · Clock.hpp*

Simulation time, and the invariant that it never goes backwards. advanceTo() is the only door, which is why the class exists rather than a bare double.

**`SimTime now() const`**
The current simulation time.

**`void advanceTo(SimTime t)`**
Moves time forward. Refuses to go backwards -- the invariant the class exists to protect.

**`void reset()`**
Back to the t=0 condition. Configuration survives; run state does not.

## Entity

*class · Entity.hpp*

The thing that flows through the model: a customer, a part, a patient. Carries an id, a creation time, a type name, and an open-ended bag of numeric attributes.

**`EntityId id() const`**
The unique id assigned at creation.

**`SimTime creationTime() const`**
When this entity entered the system. Copies inherit the original's, because they are the same work.

**`void setAttribute(const std::string& name, double value)`**
Inserts or overwrites an attribute.

**`double attribute(const std::string& name) const`**
Reads a numeric attribute. Returns 0.0 when absent, which is why callers that care check hasAttribute() first.

**`bool hasAttribute(const std::string& name) const`**
Whether the attribute has ever been set on this entity.

**`void copyAttributesFrom(const Entity& other)`**
Copies every attribute from another entity. Used when duplicating.

**`void addMember(Entity* e)`**
Adds an entity to this one's batch. A batch representative carries its members until a Separate splits them out.

**`const std::vector<Entity*>& members() const`**
The entities this batch representative is carrying.

**`bool isBatch() const`**
Whether this entity is carrying members.

**`void clearMembers()`**
Drops the carried members, after a Separate has released them.

**`void setCreationTime(SimTime t)`**
Overrides the creation time. Used when a copy must inherit the original's, because it is the same work.

**`const std::string& type() const`**
The entity's type name, which per-type reporting is keyed on.

**`void setType(std::string t)`**
Changes the type name. Prefer NodeContext::setEntityType, which also moves the live per-type count.

## EventNotice

*class · EventNotice.hpp*

One scheduled future occurrence: a time, a kind, and optionally the entity and block it concerns. Ordered by time, with a sequence number breaking ties so equal-time events keep their scheduling order.

**`EventType type() const`**
The type.

**`SimTime time() const`**
The time.

**`Entity* entity() const`**
The entity.

**`INode* node() const`**
The node.

**`uint64_t sequenceNumber() const`**
A monotonically increasing stamp used only for tie-breaking.

**`bool operator>(const EventNotice& other) const`**
Orders notices for the heap: earlier times first, with the sequence number breaking ties so equal-time events keep scheduling order.

## FutureEventList

*class · FutureEventList.hpp*

The pending-event set, kept as a binary heap ordered by time. popImminent() is the beating heart of the run loop. It cannot remove an arbitrary element, which is why reneging uses lazy cancellation instead.

**`bool isEmpty() const`**
Whether anything is still scheduled. The run loop stops when this is true.

**`std::size_t size() const`**
How many events are pending.

**`void clear()`**
Discards every pending event. Part of resetting between replications.

**`void schedule(EventNotice e)`**
Inserts a notice into the heap.

**`EventNotice popImminent()`**
Removes and returns the earliest notice. The centre of the run loop.

**`SimTime nextEventTime() const`**
When the next event is due, without removing it.

**`const EventNotice& peekImminent() const`**
The peek imminent.

## Activity

*class · Activity.hpp*

A span whose duration is known when it starts -- a service time drawn at seize. endTime() is derived, never stored, so the two cannot disagree.

**`Activity(std::string name, SimTime start, SimTime duration)`**
Constructor.

**`const std::string& name() const`**
The name this object was constructed with.

**`SimTime startTime() const`**
The start time.

**`SimTime duration() const`**
The duration.

**`SimTime endTime() const`**
Start plus duration, derived rather than stored so the two cannot disagree.

## Delay

*class · Delay.hpp*

A span whose duration is decided later, by the system rather than by a draw -- how long an entity waited for a server to free up.

**`explicit Delay(SimTime start)`**
Constructor.

**`SimTime startTime() const`**
The start time.

**`bool hasEnded() const`**
Whether this delay has been closed.

**`void end(SimTime t)`**
Closes the delay at the given time.

**`SimTime duration() const`**
The duration.

## SystemState

*class · SystemState.hpp*

A snapshot of the whole system: number in system, number queued, server status. Summed across stations rather than owned by any one of them.

**`int numberInSystem() const`**
The number in system.

**`int numberInQueue() const`**
The number in queue.

**`ResourceState serverStatus() const`**
Whether servers are idle, busy or fully seized.

**`void setNumberInSystem(int count)`**
Sets the number in system.

**`void setNumberInQueue(int count)`**
Sets the number in queue.

**`void setServerStatus(ResourceState status)`**
Sets the server status.

**`void reset()`**
Back to the t=0 condition. Configuration survives; run state does not.

## Statistics

*class · Statistics.hpp*

The accumulators behind every reported average. Holds counts, totals and two time integrals, and knows the difference between a tally average and a time-weighted one.

**`Statistics(std::string labelA, std::string labelB)`**
Constructor.

**`int numberArrived() const`**
How many arrivals were counted.

**`int numberServed() const`**
How many departures were counted.

**`SimTime totalWaitingTime() const`**
The total waiting time.

**`SimTime totalTimeInSystem() const`**
The total time in system.

**`SimTime maxWaitingTime() const`**
The max waiting time.

**`SimTime areaA() const`**
The raw first integral. Divide by measured time for the average.

**`SimTime areaB() const`**
The raw second integral. Divide by measured time for the average.

**`SimTime lastUpdateTime() const`**
When the integrals were last closed.

**`const std::string& labelA() const`**
The label a.

**`const std::string& labelB() const`**
The label b.

**`void recordArrival(SimTime t)`**
Counts an arrival.

**`void recordDeparture(SimTime t, SimTime waitTime, SimTime timeInSystem)`**
Counts a departure and accumulates its waiting time and time in system.

**`void updateTimeIntegrals(SimTime now, int valueA, int valueB)`**
Closes the interval that just ENDED, so it must be called before the clock moves and before any state changes, with the OLD values.

**`void reset()`**
Back to the t=0 condition. Configuration survives; run state does not.

**`void restartAt(SimTime now)`**
Throws away everything collected and starts accumulating from now. Warm-up removal in one line.

**`double averageWaitingTime() const`**
The average waiting time.

**`double averageTimeInSystem() const`**
The average time in system.

**`double timeAverageA(SimTime elapsed) const`**
The first time-weighted average over the measured interval. The caller supplies the elapsed time so that warm-up handling is visible.

**`double timeAverageB(SimTime elapsed) const`**
The second time-weighted average over the measured interval.

**`double utilisation(SimTime elapsed, int capacity) const`**
Busy server-time divided by capacity times elapsed time.

## ModelError

*class · ModelError.hpp · inherits public std::runtime_error*

Thrown when a model is wired wrongly -- a programmer's mistake, where stopping is the right response. Contrast Diagnostic, which is how a user's mistake is reported.

**`explicit ModelError(const std::string& what) : std::runtime_error(what)`**
Constructor.

Chapter 3

# Queues and Resources

Waiting lines, the disciplines that order them, and the servers they wait for.

A queue holds entities; a rule decides which one leaves next; a resource is the pool of servers they are waiting for. Resources are owned by the **Model** rather than by a block, which is what makes one operator shared across three machines expressible at all.

## EntityQueue

*class · EntityQueue.hpp*

A waiting line: a deque of entities plus a pluggable rule that chooses which one leaves next. Tracks its own maximum observed length.

**`EntityQueue(const std::string& name, QueueDiscipline discipline)`**
Constructor.

**`EntityQueue(const std::string& name, std::unique_ptr<IQueueRule> rule)`**
Constructor.

**`const std::string& name() const`**
The name this object was constructed with.

**`std::string ruleName() const`**
The discipline's name, for reports.

**`std::size_t length() const`**
The length.

**`bool isEmpty() const`**
Whether anything is waiting here.

**`std::size_t maxLengthObserved() const`**
The longest this queue has ever been.

**`const std::deque<Entity*>& contents() const`**
The waiting entities, in arrival order, whichever discipline is in use.

**`void setRandomStream(RandomStream* rng)`**
Supplies the stream the Random discipline draws from.

**`void reset()`**
Back to the t=0 condition. Configuration survives; run state does not.

**`void resetStatistics()`**
Discards measurements collected so far and starts again from now, without touching what is held. This is warm-up removal at block level.

**`void push(Entity* e)`**
Adds an entity to the back of the line.

**`Entity* pop()`**
Removes and returns the entity the discipline selects.

**`bool remove(Entity* e)`**
Removes a specific entity if it is still waiting, and reports whether it was. How reneging works.

## IQueueRule

*class · QueueRule.hpp*

The interface a queue discipline implements. One method: given the waiting entities, which index is served next.

**`virtual std::string name() const = 0`**
The name this object was constructed with.

## FifoRule

*class · QueueRule.hpp · inherits public IQueueRule*

First in, first out. Always index 0.

**`std::size_t selectIndex(const std::deque<Entity*>&, RandomStream&) const override`**
Which waiting entity is served next, as an index into the queue.

**`std::string name() const override`**
The name this object was constructed with.

## LifoRule

*class · QueueRule.hpp · inherits public IQueueRule*

Last in, first out. Always the final index.

**`std::size_t selectIndex(const std::deque<Entity*>&, RandomStream&) const override`**
Which waiting entity is served next, as an index into the queue.

**`std::string name() const override`**
The name this object was constructed with.

## HighestAttributeRule

*class · QueueRule.hpp · inherits public IQueueRule*

Serves whichever waiting entity has the largest value of a named attribute -- the Priority discipline. Ties resolve to the earliest arrival, deliberately.

**`explicit HighestAttributeRule(std::string attribute)`**
Constructor.

**`std::size_t selectIndex(const std::deque<Entity*>&, RandomStream&) const override`**
Which waiting entity is served next, as an index into the queue.

**`std::string name() const override`**
The name this object was constructed with.

## LowestAttributeRule

*class · QueueRule.hpp · inherits public IQueueRule*

Serves the smallest value of a named attribute -- shortest processing time, or earliest due date.

**`explicit LowestAttributeRule(std::string attribute)`**
Constructor.

**`std::size_t selectIndex(const std::deque<Entity*>&, RandomStream&) const override`**
Which waiting entity is served next, as an index into the queue.

**`std::string name() const override`**
The name this object was constructed with.

## RandomRule

*class · QueueRule.hpp · inherits public IQueueRule*

Picks a waiting entity uniformly at random, drawing from the queue's random stream.

**`std::size_t selectIndex(const std::deque<Entity*>&, RandomStream&) const override`**
Which waiting entity is served next, as an index into the queue.

**`std::string name() const override`**
The name this object was constructed with.

## IResourceUser

*class · Resource.hpp*

The interface a block implements to compete for a shared resource: can it start something, and how long has its longest-waiting entity been waiting.

**`virtual bool hasWaiting() const = 0`**
Whether waiting.

**`virtual SimTime headOfLineSince() const = 0`**
When this user's longest-waiting entity started waiting. The resource compares this across every block sharing it, so the longest waiter anywhere is served next.

**`virtual void startFromQueue(NodeContext& ctx) = 0`**

Asks this user to start serving its next queued entity, if it can. Returning without seizing tells the resource to stop offering.

## Resource

*class · Resource.hpp*

A pool of interchangeable servers with a capacity. Owned by the Model, not by a block, which is what lets several blocks share one operator. When a unit frees, the resource offers it to whichever user has waited longest.

**`Resource(std::string name, int capacity)`**
Constructor.

**`const std::string& name() const`**
The name this object was constructed with.

**`int capacity() const`**
How many units this resource has.

**`int unitsBusy() const`**
The units busy.

**`int unitsAvailable() const`**
Capacity minus units busy, derived rather than stored.

**`bool isAvailable() const`**
Whether at least one unit is free.

**`ResourceState state() const`**
The state.

**`void reset()`**
Back to the t=0 condition. Configuration survives; run state does not.

**`void addUser(IResourceUser* user)`**
Registers a block as a competitor for this resource.

**`std::size_t userCount() const`**
How many blocks compete for this resource.

**`void offerFreedUnit(NodeContext& ctx)`**
Offers a freed unit to whichever registered user has the longest-waiting entity, so a shared resource does not favour the block that released it.

**`void seize(int units = 1)`**
Takes units. Asserts they are available; the caller must check first.

**`void release(int units = 1)`**
Returns units to the pool.

Chapter 4

# Randomness

One primitive, twelve distributions, and the tests that show a generator is sound.

Everything random is built on a single u01() call, so a run is reproducible from one seed. Distributions may be given their own named streams, which is what lets you change a service time without the arrival pattern shifting underneath you.

## EngineKind

*enum class · RandomStream.hpp*

Which underlying generator a RandomStream uses. Includes RANDU, kept precisely so its failure can be demonstrated.

**Values** MersenneTwister, LinearCongruential, Randu

## RandomStream

*class · RandomStream.hpp*

The single source of randomness. Everything is built on one u01() primitive, so the whole engine is reproducible from one seed. Supports named substreams and antithetic variates.

**`unsigned seed() const`**
The seed.

**`EngineKind engine() const`**
The engine.

**`long long draws() const`**
The draws.

**`void reset()`**
Back to the t=0 condition. Configuration survives; run state does not.

**`double u01()`**
The single primitive. Every distribution in the engine is built on this one call.

**`void setAntithetic(bool on)`**
Makes the stream return 1-u instead of u, the basis of antithetic variates.

**`bool isAntithetic() const`**
Whether antithetic.

**`RandomStream substream(const std::string& name) const`**
A named, independent stream derived from the base seed. Naming is what keeps a stream stable as the rest of a model changes.

**`SimTime exponential(SimTime mean)`**
One exponential sample with the given MEAN. The inversion is the only place the 1/lambda conversion happens.

**`SimTime uniform(SimTime low, SimTime high)`**
One uniform sample on a closed interval.

**`SimTime normal(SimTime mean, SimTime stdDev)`**
One normal sample, by the Box-Muller transform.

**`SimTime lognormal(SimTime logMean, SimTime logStdDev)`**
One lognormal sample, from the mean and sd of the LOGARITHM.

**`SimTime weibull(SimTime scale, SimTime shape)`**
One Weibull sample, by inversion.

**`SimTime erlang(SimTime meanOfEach, int phases)`**
One Erlang sample: the sum of `phases` exponential draws.

**`SimTime triangular(SimTime low, SimTime mode, SimTime high)`**
One triangular sample, by inversion either side of the mode.

**`int poisson(double mean)`**
One Poisson count, by Knuth's multiplication method.

**`bool bernoulli(double p)`**
True with probability p. One draw.

**`std::size_t uniformIndex(std::size_t n)`**
A uniformly chosen index in [0, n). What the Random queue discipline uses.

**`static double normalQuantile(double p)`**
The inverse normal CDF, used to build normal samples and confidence intervals.

## IDistribution

*class · Distribution.hpp*

The interface every distribution implements: draw a value, report a theoretical mean, describe yourself, clone yourself, and optionally sample from your own stream.

**`virtual SimTime draw(RandomStream& rng) = 0`**
Samples one value, using this object's own stream if it has been given one.

**`virtual std::string describe() const = 0`**
A human-readable one-line description, used in model dumps and traces.

**`virtual void reset()`**
Back to the t=0 condition. Configuration survives; run state does not.

**`virtual SimTime mean() const = 0`**
The theoretical mean, used by the stability check before the run.

**`virtual std::unique_ptr<IDistribution> clone() const = 0`**
A deep copy. Needed because a distribution may be held in two places at once.

**`void useStream(RandomStream* s)`**
Gives every sampling site beneath this node its own random stream.

**`RandomStream* stream() const`**
The stream.

## Exponential

*class · Distribution.hpp · inherits public IDistribution*

Exponential inter-event times, parameterised by MEAN, not rate. The 1/lambda conversion happens in exactly one line.

**`explicit Exponential(SimTime mean)`**
Constructor.

**`SimTime draw(RandomStream& rng) override`**
Samples one value, using this object's own stream if it has been given one.

**`std::string describe() const override`**
A human-readable one-line description, used in model dumps and traces.

**`SimTime mean() const override`**
The theoretical mean, used by the stability check before the run.

**`std::unique_ptr<IDistribution> clone() const override`**
A deep copy. Needed because a distribution may be held in two places at once.

## Constant

*class · Distribution.hpp · inherits public IDistribution*

Always the same value. Consumes nothing from the random stream, which matters when comparing runs.

**`explicit Constant(SimTime value)`**
Constructor.

**`SimTime draw(RandomStream& rng) override`**
Samples one value, using this object's own stream if it has been given one.

**`std::string describe() const override`**
A human-readable one-line description, used in model dumps and traces.

**`SimTime mean() const override`**
The theoretical mean, used by the stability check before the run.

**`std::unique_ptr<IDistribution> clone() const override`**
A deep copy. Needed because a distribution may be held in two places at once.

## Uniform

*class · Distribution.hpp · inherits public IDistribution*

Uniform on a closed interval.

**`Uniform(SimTime low, SimTime high)`**
Constructor.

**`SimTime draw(RandomStream& rng) override`**
Samples one value, using this object's own stream if it has been given one.

**`std::string describe() const override`**
A human-readable one-line description, used in model dumps and traces.

**`SimTime mean() const override`**
The theoretical mean, used by the stability check before the run.

**`std::unique_ptr<IDistribution> clone() const override`**
A deep copy. Needed because a distribution may be held in two places at once.

## Triangular

*class · Distribution.hpp · inherits public IDistribution*

Triangular with a low, a mode and a high. The usual choice when you have an expert's best, worst and likely case and no data.

**`Triangular(SimTime low, SimTime mode, SimTime high)`**
Constructor.

**`SimTime draw(RandomStream& rng) override`**
Samples one value, using this object's own stream if it has been given one.

**`std::string describe() const override`**
A human-readable one-line description, used in model dumps and traces.

**`SimTime mean() const override`**
The theoretical mean, used by the stability check before the run.

**`std::unique_ptr<IDistribution> clone() const override`**
A deep copy. Needed because a distribution may be held in two places at once.

## Normal

*class · Distribution.hpp · inherits public IDistribution*

Normal, optionally truncated at zero so a duration cannot come out negative.

**`Normal(SimTime mean, SimTime stdDev, bool truncateAtZero = true)`**
Constructor.

**`SimTime draw(RandomStream& rng) override`**
Samples one value, using this object's own stream if it has been given one.

**`std::string describe() const override`**
A human-readable one-line description, used in model dumps and traces.

**`SimTime mean() const override`**
The theoretical mean, used by the stability check before the run.

**`std::unique_ptr<IDistribution> clone() const override`**
A deep copy. Needed because a distribution may be held in two places at once.

## Lognormal

*class · Distribution.hpp · inherits public IDistribution*

Lognormal. Its constructor takes the mean and standard deviation of the LOGARITHM; use fromMeanAndSd() if you have the observed values instead.

**`Lognormal(SimTime logMean, SimTime logStdDev)`**
Constructor.

**`static std::unique_ptr<Lognormal> fromMeanAndSd(SimTime mean, SimTime sd)`**
Builds a Lognormal from the mean and standard deviation you observed, rather than those of the logarithm. The conversion is not obvious and getting it wrong is silent.

**`SimTime draw(RandomStream& rng) override`**
Samples one value, using this object's own stream if it has been given one.

**`std::string describe() const override`**
A human-readable one-line description, used in model dumps and traces.

**`SimTime mean() const override`**
The theoretical mean, used by the stability check before the run.

**`std::unique_ptr<IDistribution> clone() const override`**
A deep copy. Needed because a distribution may be held in two places at once.

## Weibull

*class · Distribution.hpp · inherits public IDistribution*

Weibull, by scale and shape. Common for time-to-failure.

**`Weibull(SimTime scale, SimTime shape)`**
Constructor.

**`SimTime draw(RandomStream& rng) override`**
Samples one value, using this object's own stream if it has been given one.

**`std::string describe() const override`**
A human-readable one-line description, used in model dumps and traces.

**`SimTime mean() const override`**
The theoretical mean, used by the stability check before the run.

**`std::unique_ptr<IDistribution> clone() const override`**
A deep copy. Needed because a distribution may be held in two places at once.

## Erlang

*class · Distribution.hpp · inherits public IDistribution*

A sum of k exponential phases. fromMean() takes the mean you actually want.

**`Erlang(SimTime meanOfEachPhase, int phases)`**
Constructor.

**`static std::unique_ptr<Erlang> fromMean(SimTime totalMean, int phases)`**
Builds an Erlang from the total mean you want and a phase count.

**`SimTime draw(RandomStream& rng) override`**
Samples one value, using this object's own stream if it has been given one.

**`std::string describe() const override`**
A human-readable one-line description, used in model dumps and traces.

**`SimTime mean() const override`**
The theoretical mean, used by the stability check before the run.

**`std::unique_ptr<IDistribution> clone() const override`**
A deep copy. Needed because a distribution may be held in two places at once.

## Discrete

*class · Distribution.hpp · inherits public IDistribution*

A finite set of values with probabilities. Takes INDIVIDUAL probabilities summing to one; Arena's DISC spelling takes cumulative ones and is converted at the grammar boundary.

**`Discrete(std::vector<SimTime> values, std::vector<double> probabilities)`**
Constructor.

**`SimTime draw(RandomStream& rng) override`**
Samples one value, using this object's own stream if it has been given one.

**`std::string describe() const override`**
A human-readable one-line description, used in model dumps and traces.

**`SimTime mean() const override`**
The theoretical mean, used by the stability check before the run.

**`std::unique_ptr<IDistribution> clone() const override`**
A deep copy. Needed because a distribution may be held in two places at once.

## Empirical

*class · Distribution.hpp · inherits public IDistribution*

Samples straight from observed data with linear interpolation between order statistics -- for when you have a hundred real service times and no idea which textbook curve produced them.

**`explicit Empirical(std::vector<SimTime> observations)`**
Constructor.

**`SimTime draw(RandomStream& rng) override`**
Samples one value, using this object's own stream if it has been given one.

**`std::string describe() const override`**
A human-readable one-line description, used in model dumps and traces.

**`SimTime mean() const override`**
The theoretical mean, used by the stability check before the run.

**`std::unique_ptr<IDistribution> clone() const override`**
A deep copy. Needed because a distribution may be held in two places at once.

## Poisson

*class · Distribution.hpp · inherits public IDistribution*

Counts rather than durations: arrivals per hour, defects per batch.

**`explicit Poisson(double mean)`**
Constructor.

**`SimTime draw(RandomStream& rng) override`**
Samples one value, using this object's own stream if it has been given one.

**`std::string describe() const override`**
A human-readable one-line description, used in model dumps and traces.

**`SimTime mean() const override`**
The theoretical mean, used by the stability check before the run.

**`std::unique_ptr<IDistribution> clone() const override`**
A deep copy. Needed because a distribution may be held in two places at once.

## Deterministic

*class · Distribution.hpp · inherits public IDistribution*

Walks a fixed list of values, optionally repeating. The basis of every hand-checkable test in this project, because it makes a run reproducible on paper.

**`explicit Deterministic(std::vector<SimTime> values, bool repeat = true)`**
Constructor.

**`SimTime draw(RandomStream& rng) override`**
Samples one value, using this object's own stream if it has been given one.

**`std::string describe() const override`**
A human-readable one-line description, used in model dumps and traces.

**`SimTime mean() const override`**
The theoretical mean, used by the stability check before the run.

**`std::unique_ptr<IDistribution> clone() const override`**
A deep copy. Needed because a distribution may be held in two places at once.

**`void reset() override`**
Back to the t=0 condition. Configuration survives; run state does not.

## TestResult

*struct · StreamTests.hpp*

The outcome of one generator-quality test: a statistic, a threshold, and whether it passed.

## StreamTests

*class · StreamTests.hpp*

Statistical tests a random stream should survive: chi-square uniformity, Kolmogorov-Smirnov, runs up and down, autocorrelation, and a 3-D serial test that exposes lattice structure.

**`static TestResult chiSquareUniformity(RandomStream& rng, int n, int bins = 10)`**
Bins many draws and compares the counts against a flat expectation. Catches a generator whose output is not evenly spread.

**`static TestResult kolmogorovSmirnov(RandomStream& rng, int n)`**
Compares the empirical distribution against the uniform one by their largest gap. Sensitive where chi-square is not.

**`static TestResult runsUpAndDown(RandomStream& rng, int n)`**
Counts ascending and descending runs against their expected number. Catches sequences that are uniform but not independent.

**`static TestResult autocorrelation(RandomStream& rng, int n, int lag = 1)`**
Correlation between draws a given lag apart. A generator can pass every one-dimensional test and still fail this.

**`static TestResult serial3D(RandomStream& rng, int n, int binsPerAxis = 16)`**
Bins consecutive triples in the unit cube. This is the test that exposes RANDU, whose triples fall on fifteen planes.

**`static std::vector<TestResult> runAll(RandomStream& rng, int n = 600000)`**
Runs every test and collects the results.

**`static void report(const std::vector<TestResult>& results, const std::string& title)`**
Prints a table of test results.

**`static double chiSquareCritical95(int degreesOfFreedom)`**
The 95% critical value of chi-square, so a test can report pass or fail rather than a bare statistic.

Chapter 5

# The Expression Layer

How a model's conditions, values and durations became text rather than compiled code.

Through v9 a service time was a constructed distribution and a condition was a lambda, so changing a model needed a compiler. v10 makes every one of those fields a string. There is no separate distribution field any more: EXPO(0.8) is a function in the grammar whose node owns one of the twelve IDistribution objects that already existed.

**Value**

A variant of double and string: what an expression evaluates to. Strings exist for one purpose, comparing an entity type. Booleans are numbers, 0 and 1, which is what lets a condition field and a duration field be the same kind of field.

**Errors are data, not exceptions**

A malformed expression is a person typing in a cell, not a programmer bug, so the lexer and parser never throw: they record a Diagnostic carrying a SourceSpan, recover, and keep going. One call reports many problems. The exception is expr(), which throws, because on the C++ API a bad expression really is a programmer error and there is no cell to point at.

## ExpressionError

*class · Value.hpp · inherits public ModelError*

Thrown when an expression cannot do its job AT RUN TIME -- a string in arithmetic, division by zero, an attribute read with no entity.

**`explicit ExpressionError(const std::string& what) : ModelError(what)`**
Constructor.

## SourceSpan

*struct · Diagnostic.hpp*

An offset and length within the text that was parsed. Carrying this from the lexer to the diagnostic is what lets a front end put a cursor on the exact bad character.

## Severity

*enum class · Diagnostic.hpp*

Whether a diagnostic is an error or a warning.

**Values** Error, Warning

## Diagnostic

*struct · Diagnostic.hpp*

A user's mistake, reported as DATA rather than thrown. The parser collects a list of these and keeps going, because a spreadsheet must show every bad cell at once and an exception can only carry one failure.

## TokenKind

*enum class · Lexer.hpp*

The lexical categories of the expression grammar.

**Values** Number, Text, Identifier, Plus, Minus, Star, Slash, Percent, Caret, Bang, Less, LessEqual, Greater, GreaterEqual, EqualEqual, BangEqual, AndAnd, OrOr, LParen, RParen, Comma, End, Invalid

## Token

*struct · Lexer.hpp*

One lexed token: its kind, its span, and its number or text.

## LexResult

*struct · Lexer.hpp*

Everything one call to tokenise() produced: the tokens, always ending in End, and any diagnostics.

## UnaryOp

*enum class · Expression.hpp*

The prefix operators: negation and logical not.

**Values** Negate, Not

## BinaryOp

*enum class · Expression.hpp*

The infix operators, from arithmetic through comparison to the short-circuiting logicals.

**Values** Add, Subtract, Multiply, Divide, Modulo, Power, Less, LessEqual, Greater, GreaterEqual, Equal, NotEqual, And, Or

## IExpression

*class · Expression.hpp*

A node in the parsed expression tree. Beyond evaluate(), the interesting members are validate(), which reports problems without throwing, and meanIfKnown(), which admits when a mean cannot be computed in advance.

**`explicit IExpression(SourceSpan span) : m_span(span)`**
Constructor.

**`SourceSpan span() const`**
The range of source text this came from.

**`virtual Value evaluate(EvalContext& ctx) const = 0`**
Computes this node's value in the given context.

**`virtual void validate(const ValidationContext& vc, std::vector<Diagnostic>& out) const = 0`**
Appends every problem found. Never throws, so one call reports many.

**`virtual std::optional<double> meanIfKnown() const`**
The mean if it can be computed in advance, or nothing. Nothing means 'cannot tell', never 'zero'.

**`virtual std::string describe() const = 0`**
A human-readable one-line description, used in model dumps and traces.

**`virtual std::unique_ptr<IExpression> clone() const = 0`**
A deep copy of the whole subtree.

**`virtual void useStream(RandomStream* s)`**
Gives every sampling site beneath this node its own random stream.

**`virtual void reset()`**
Back to the t=0 condition between replications. Only a Deterministic distribution has anything to do, but an expression that owns one must pass it on.

## LiteralExpression

*class · Expression.hpp · inherits public IExpression*

A number or a quoted string, known at parse time.

**`LiteralExpression(Value v, SourceSpan span) : IExpression(span), m_value(std::move(v))`**
Constructor.

**`const Value& value() const`**
The value.

**`Value evaluate(EvalContext&) const override`**
Computes this node's value in the given context.

**`void validate(const ValidationContext&, std::vector<Diagnostic>&) const override`**
Appends every problem found to the list. Does not throw and does not stop at the first.

**`std::optional<double> meanIfKnown() const override`**
The mean if it can be computed in advance, or nothing. Nothing means 'cannot tell', never 'zero'.

**`std::string describe() const override`**
A human-readable one-line description, used in model dumps and traces.

**`ExpressionPtr clone() const override`**
A deep copy. Needed because a distribution may be held in two places at once.

## NameExpression

*class · Expression.hpp · inherits public IExpression*

A bare name, resolved LATE -- the parser stores only the text, and whether it is an attribute, a variable, Entity.Type or TNOW is decided at validation and evaluation. That is what stops the parser going stale as new kinds of name are added.

**`const std::string& name() const`**
The text the parser captured, still unresolved.

**`Value evaluate(EvalContext& ctx) const override`**
Computes this node's value in the given context.

**`void validate(const ValidationContext& vc, std::vector<Diagnostic>& out) const override`**
Appends every problem found to the list. Does not throw and does not stop at the first.

**`std::string describe() const override`**
A human-readable one-line description, used in model dumps and traces.

**`ExpressionPtr clone() const override`**
A deep copy. Needed because a distribution may be held in two places at once.

## UnaryExpression

*class · Expression.hpp · inherits public IExpression*

A prefix operator applied to one operand.

**`Value evaluate(EvalContext& ctx) const override`**
Computes this node's value in the given context.

**`void validate(const ValidationContext& vc, std::vector<Diagnostic>& out) const override`**
Appends every problem found to the list. Does not throw and does not stop at the first.

**`std::optional<double> meanIfKnown() const override`**
The mean if it can be computed in advance, or nothing. Nothing means 'cannot tell', never 'zero'.

**`std::string describe() const override`**
A human-readable one-line description, used in model dumps and traces.

**`ExpressionPtr clone() const override`**
A deep copy. Needed because a distribution may be held in two places at once.

**`void useStream(RandomStream* s) override`**
Gives every sampling site beneath this node its own random stream.

**`void reset() override`**
Back to the t=0 condition. Configuration survives; run state does not.

## BinaryExpression

*class · Expression.hpp · inherits public IExpression*

An infix operator over two operands. The logical operators short-circuit, which is what makes a guarded division safe to write.

**`Value evaluate(EvalContext& ctx) const override`**
Computes this node's value in the given context.

**`void validate(const ValidationContext& vc, std::vector<Diagnostic>& out) const override`**
Appends every problem found to the list. Does not throw and does not stop at the first.

**`std::string describe() const override`**
A human-readable one-line description, used in model dumps and traces.

**`ExpressionPtr clone() const override`**
A deep copy. Needed because a distribution may be held in two places at once.

**`void useStream(RandomStream* s) override`**
Gives every sampling site beneath this node its own random stream.

**`void reset() override`**
Back to the t=0 condition. Configuration survives; run state does not.

## LambdaExpression

*class · Expression.hpp · inherits public IExpression*

Adapter wrapping a C++ predicate so the pre-v10 std::function API is a constructor for the one representation rather than a second code path.

**`Value evaluate(EvalContext& ctx) const override`**
Computes this node's value in the given context.

**`void validate(const ValidationContext&, std::vector<Diagnostic>&) const override`**
Appends every problem found to the list. Does not throw and does not stop at the first.

**`std::string describe() const override`**
A human-readable one-line description, used in model dumps and traces.

**`ExpressionPtr clone() const override`**
A deep copy. Needed because a distribution may be held in two places at once.

## DistributionExpression

*class · Expression.hpp · inherits public IExpression*

Adapter wrapping an IDistribution so the pre-v10 API keeps working. Also what a parsed EXPO(0.8) with constant arguments becomes.

**`Value evaluate(EvalContext& ctx) const override`**
Computes this node's value in the given context.

**`void validate(const ValidationContext&, std::vector<Diagnostic>&) const override`**
Appends every problem found to the list. Does not throw and does not stop at the first.

**`std::optional<double> meanIfKnown() const override`**
The mean if it can be computed in advance, or nothing. Nothing means 'cannot tell', never 'zero'.

**`std::string describe() const override`**
A human-readable one-line description, used in model dumps and traces.

**`ExpressionPtr clone() const override`**
A deep copy. Needed because a distribution may be held in two places at once.

**`void useStream(RandomStream* s) override`**
Gives every sampling site beneath this node its own random stream.

**`void reset() override`**
Back to the t=0 condition. Configuration survives; run state does not.

## ParseResult

*struct · Parser.hpp*

What parsing produced: a tree, and the diagnostics found on the way. The tree is never null; on error it holds a placeholder that is never evaluated.

## IModelState

*class · EvalContext.hpp*

The four questions an expression may ask about a running system, plus the clock. Declared by the expression layer and implemented by SimulationSystem, so the expression layer never has to include Model.hpp.

**`virtual double queueLength(const std::string& blockName) const = 0`**
How many entities are waiting at a named block. Throws on an unknown name rather than reading as an empty queue.

**`virtual double resourceBusy(const std::string& name) const = 0`**
Units of a named resource currently seized.

**`virtual double resourceCapacity(const std::string& name) const = 0`**
A named resource's capacity.

**`virtual double numberInSystem() const = 0`**
How many entities are anywhere in the system.

**`virtual SimTime now() const = 0`**
The current simulation time.

## EvalContext

*class · EvalContext.hpp*

The only view of the engine an expression gets while a run is happening: the current entity, the variables, the model state and the random stream. Any of them may be absent, and each accessor says so rather than returning a plausible default.

**`const Entity* entity() const`**
The entity.

**`VariableStore* variables() const`**
The variables.

**`const IModelState* state() const`**
The state.

**`RandomStream* rng() const`**
The rng.

**`double attribute(const std::string& name) const`**
Reads an entity attribute, throwing if there is no entity rather than returning zero.

**`const std::string& entityType() const`**
The current entity's type name.

**`RandomStream& requireRng() const`**
The random stream, or an error explaining that none was supplied.

**`const IModelState& requireState() const`**
The live model state, or an error explaining that none was supplied.

## FieldContext

*enum class · EvalContext.hpp*

Whether the field being validated has an entity to read from. A Create block's interarrival time is evaluated before its entity exists, so an attribute reference there is an error caught before the run.

**Values** HasEntity, NoEntity

## ValidationContext

*struct · EvalContext.hpp*

What a name is allowed to resolve to, checked before the run: the declared variables and attributes, and whether an entity is available.

**`bool isVariable(const std::string& name) const`**
Whether this name is a declared variable.

**`bool isAttribute(const std::string& name) const`**
Whether this name is a declared attribute.

## VariableStore

*class · VariableStore.hpp*

Arena's Variable data module. Global, numeric, time-persistent values that belong to the system rather than to any entity. A variable may not share a name with an attribute, and writing an undeclared one throws.

**`void declare(const std::string& name, double initialValue)`**
Declares a variable with an initial value. Refuses a duplicate, and refuses a name already used by an attribute.

**`bool isArray(const std::string& name) const`**
Whether array.

**`int rowsOf(const std::string& name) const`**
How many rows an array has; 0 for a name that is not an array.

**`int columnsOf(const std::string& name) const`**
How many columns an array has; 0 for a name that is not an array.

**`double element(const std::string& name, double row, double column = 1.0) const`**
One cell, 1-based and rounded. Outside the table is a ModelError naming the cell and the table's size, never a read of whatever is next in memory.

**`std::vector<std::string> arrayNames() const`**
The array names.

**`void noteAttributeNames(std::vector<std::string> names)`**
Supplies the attribute names that declare() checks against.

**`bool has(const std::string& name) const`**
Whether a variable of this name exists.

**`double get(const std::string& name) const`**
The current value.

**`void set(const std::string& name, double value, SimTime now)`**
Closes the time integral at `now`, then changes the value. Throws on an undeclared name, so a typo cannot become a second variable.

**`void reset()`**
Back to the t=0 condition. Configuration survives; run state does not.

**`void resetStatistics(SimTime now)`**
Discards measurements collected so far and starts again from now, without touching what is held. This is warm-up removal at block level.

**`void updateIntegrals(SimTime now)`**
Closes every variable's integral up to now.

**`double timeAverage(const std::string& name) const`**
The time-weighted average since measurement began.

**`const std::vector<std::string>& names() const`**
The variables, in declaration order.

**`std::size_t count() const`**
How many entities have passed through.

## ArrayReadExpression

*class · Expression.hpp · inherits public IExpression*

Name(i) or Name(i, j) on an array Variable. The parser builds one only for a name it was told is an array, so a mistyped function is still reported as an unknown function rather than read as a table.

**`Value evaluate(EvalContext& ctx) const override`**
Computes this node's value in the given context.

**`void validate(const ValidationContext& vc, std::vector<Diagnostic>& out) const override`**
Appends every problem found to the list. Does not throw and does not stop at the first.

**`std::string describe() const override`**
A human-readable one-line description, used in model dumps and traces.

**`ExpressionPtr clone() const override`**
A deep copy. Needed because a distribution may be held in two places at once.

**`void useStream(RandomStream* s) override`**
Gives every sampling site beneath this node its own random stream.

**`void reset() override`**
Back to the t=0 condition. Configuration survives; run state does not.

Chapter 6

# The Document Layer

How a model stopped being a program and became a set of spreadsheets.

v10 made a model's fields text; v11 makes the model itself data. Every module type publishes its columns through ModuleRegistry, a ModelDocument holds rows of those cells as text, the .des reader and writer round-trip them byte for byte, and compile() turns a document into a Model or into diagnostics that point at individual cells. The engine never learns what a document is.

### Why a document is not a Model

A Model must always be valid: a route names a block that exists, a Process names a resource that was declared. A document being edited is none of those things - it is half-finished and contradictory most of the time it is open. Two types, therefore, and one direction of travel between them.

### Rows are addressed by position

Never by a stable id, because the two disagree the moment a row moves - and row order is semantic here. A Decide takes the first branch that matches and an Assign runs its fields in order, so reordering two rows changes what the model does.

### Four passes, all of which run

Schema, then references, then expressions, then structure. Every pass runs even when an earlier one found errors, because the point of the layer is to report every bad cell rather than the first. The structure pass is the one exception: it cannot run with broken references, so it is skipped and structureChecked says so.

## ColumnType

*enum class · ModuleSchema.hpp*

What kind of value a column holds: Text, Identifier, Integer, Real, Boolean, Enum, Expression or Reference. The schema pass checks a cell against this, and a front end picks an editor from it.

**Values** Text, Identifier, Integer, Real, Boolean, Enum, Expression, Reference

## ModuleKind

*enum class · ModuleSchema.hpp*

Whether a module is Data (a Resource, a Variable), Flowchart (a block entities pass through), or Child (a row belonging to another row, such as one branch of a Decide).

**Values** Flowchart, Data, Child

## Column

*struct · ModuleSchema.hpp*

One column of a module's spreadsheet: its name, type, whether it is required, its default, the values an Enum allows, and for a Reference the module type it must name. 'Block' is the pseudo-target meaning any flowchart block, which is what an exit column points at.

## ModuleSchema

*struct · ModuleSchema.hpp*

Everything a front end needs to render one module type: its name, its kind, its columns, whether it is read-only, and for a child table the column naming its parent. Queue is the one read-only schema, because this engine has no queue object apart from its Process.

**`const Column* column(const std::string& id) const`**
The named column, or null.

## ModuleRegistry

*class · ModuleSchema.hpp*

The published set of schemas, a singleton built once at startup. A front end asks it what module types exist and what columns each has, so a module added in a later version renders without the front end changing.

**`static const ModuleRegistry& instance()`**
The one registry. Built on first use from buildSchemas().

**`const ModuleSchema* find(const std::string& typeName) const`**
The schema for a module type, or null. Null is not an error: an unknown type is preserved and warned about, never dropped.

**`const std::vector<ModuleSchema>& all() const`**
Every schema, in the order a front end should list them.

## ModelDocument

*class · ModelDocument.hpp*

Rows of text cells, grouped by module type and addressed by POSITION. Unlike a Model it may be invalid, half-finished and contradictory, which is what makes it editable. It also keeps the source lines it was read from, so writing it back reproduces the file byte for byte.

**`const std::vector<std::string>& types() const`**
Every module type present, in the order they first appeared. The build pass follows this, so it is the order blocks are created in.

**`std::size_t rowCount(const std::string& type) const`**
How many rows a module type has.

**`const std::vector<Row>& rows(const std::string& type) const`**
Every row of a module type, in file order. Order is semantic: a Decide takes the first branch that matches, an Assign runs its fields in order.

**`Row& addRow(const std::string& type)`**
Appends an empty row to a module type's table and returns its position.

**`void removeRow(const std::string& type, std::size_t index)`**
Deletes a row by position. Every later row shifts down, which is why nothing holds a row index across an edit.

**`void moveRow(const std::string& type, std::size_t from, std::size_t to)`**
Moves a row to another position. It exists because row order is part of the model, not a display preference.

**`void setSourceLines(std::vector<std::string> lines)`**
Sets the source lines.

**`void setPreamble(std::vector<std::string> lines)`**
Sets the preamble.

**`void setTrailer(std::vector<std::string> lines)`**
Sets the trailer.

**`const std::vector<std::string>& preamble() const`**
The preamble.

**`const std::vector<std::string>& trailer() const`**
The trailer.

**`Row& rowAt(const std::string& type, std::size_t index)`**
One row, mutable, by type and position. It exists for the reader, which fills in a row source block after parsing the record.

**`const std::vector<std::string>& sourceLines() const`**
The source lines.

**`bool edited() const`**
The edited.

**`void markEdited()`**
Performs mark edited.

## ReadResult

*struct · DocumentFormat.hpp*

What reading a .des file produces: the document, the diagnostics found while reading it, and the format version the file declared. Reading never throws -- a malformed line becomes a diagnostic and reading continues.

## CellRef

*struct · Diagnostic.hpp*

Which cell a diagnostic came from: module type, row position, column name. It composes with v10's SourceSpan rather than replacing it, so the character offset the parser measured survives inside the cell the compiler names.

## CompileResult

*struct · Compiler.hpp*

What compiling a document produces: a Model when it succeeded, the diagnostics either way, and whether the structural pass actually ran. That last flag matters because the structure pass cannot run with broken references, and reporting a model as checked when it was not is the failure this project keeps refusing.

Chapter 7

# Runtime Control

How a run stopped being something you wait for and became something you drive.

SimulationSystem::run() blocks until the stopping rule is met, which is the right shape for a program and the wrong one for a front end: there is no moment at which it can redraw, and no way to stop. v12 splits that loop. stepOnce() does one event, RunController owns the loop around it and the replication loop above that, and run() becomes a loop over stepOnce() and nothing else -- the same loop, with the caller holding the handle.

### Why stepping and not threads

The engine's entire claim is that one seed gives one run. A thread buys responsiveness that a stepped loop already provides, and pays for it with a lock between the clock and the statistics and a class of bug that no byte-identical gate can reliably reproduce. Pausing is not calling; cancelling is not calling ever again.

### Watching must not change the run

A snapshot is built when it is asked for, from accessors that already existed. Nothing is added to the inner loop, so a run is bit-for-bit identical whether or not anyone was watching -- which is asserted rather than assumed, and which a watcher that consumed a single random number was shown to break.

### Progress that admits it cannot tell

ITerminationRule::progress() returns an optional, and nothing means CANNOT TELL rather than zero. A time limit divides; the drained-system rule has no override at all, because whether a system will next be empty is not knowable in advance. A bar that reads 0% for a whole run and then jumps to 100% is a lie the caller has no way to detect.

### One replication loop

RunController owns the loop over replications, and Experiment is a wrapper over it. Two loops would have had to agree about seeding, antithetic pairing, warm-up and the observation grid, and the one that drifts is always the one nobody runs. Antithetic pairing is where the abstraction leaks: one replication is two runs, so a progress fraction restarts at zero halfway through.

## RunState

*enum class · RunController.hpp*

Where a run has got to: Ready, Running, Paused, Finished, Cancelled or Failed. Finished and Cancelled are terminal, and advance() on either returns 0 rather than throwing -- a front end that polls past the end is normal, not wrong.

**Values** Ready, Running, Paused, Finished, Cancelled, Failed

## RunProgress

*struct · RunController.hpp*

What a watcher reads between calls: which replication, the clock, how many events have been processed, and a fraction that MAY BE ABSENT. Absent means the stopping rule cannot tell how far through it is, which is not the same as zero.

## BlockSnapshot

*struct · RunController.hpp*

One flowchart block at an instant: its queue length, how many it has served, and its utilisation over the measured period.

## ResourceSnapshot

*struct · RunController.hpp*

One resource at an instant: how many units are busy, out of how many it has.

## VariableSnapshot

*struct · RunController.hpp*

One declared variable and its current value.

## RunSnapshot

*struct · RunController.hpp*

The whole running system at an instant -- clock, arrivals, departures, number in system, and one entry per block, resource and variable. Built ON DEMAND from accessors that already existed, so nothing is added to the inner loop and a run is bit-for-bit identical whether or not anyone was watching.

## RunSetup

*struct · RunSetup.hpp*

How to run a model: length, warm-up, replications, base seed, and the stopping conditions. Read from the [Run] module, so a model file carries its own run length rather than depending on a number stored somewhere else.

## RunController

*class · RunController.hpp*

The run, with the loop handed to the caller. advance(budget) does at most that many events and returns; pausing is not calling, cancelling is not calling ever again. It owns the replication loop too, which is why Experiment builds on it rather than keeping a second implementation.

**`RunController(RunSetup setup, ModelBuilder build)`**
Constructor.

**`~RunController()`**
Destructor.

**`RunState state() const`**
The state.

**`const std::string& failure() const`**
Why the run Failed, when it did.

**`std::size_t advance(std::size_t maxEvents)`**
Does at most maxEvents events, rolling on to the next replication when one ends, and returns how many it actually did. Zero when paused, finished, cancelled or failed. A model error part way through a run -- SQRT of a negative number, a service time that comes out negative -- marks the run Failed with that message instead of propagating. Until 15.1 it went out through every caller, and in the terminal UI that ended the program.

**`void pause()`**
Running becomes Paused, and advance() then does nothing.

**`void resume()`**
Paused becomes Running. It cannot undo a cancel.

**`void cancel()`**
Stops for good. The run so far stays readable.

**`RunProgress progress() const`**
A RunProgress for the moment it is called.

**`RunSnapshot snapshot() const`**

A RunSnapshot of the replication in progress, or an empty one between replications.

**`void report(std::ostream& os) const`**
One replication prints the run own report; several print confidence intervals, because quoting one number out of thirty runs is what Experiment exists to stop.

**`const std::vector<ReplicationResult>& results() const`**
One ReplicationResult per completed replication.

**`const std::vector<std::vector<NamedStatistic>>& namedStatistics() const`**
The named statistics.

**`const std::vector<std::vector<double>>& series() const`**
The series.

**`void runToCompletion()`**
Drives to the end. Loops on STATE rather than on the count advance() returns: a replication that produced no events would return 0 and strand a caller that trusted that count.

## RegressionOutcome

*struct · Regression.hpp*

What happened to one model in a regression run: whether it matched, and if not, the first line where it differed.

## RegressionReport

*struct · Regression.hpp*

The verdict over a whole manifest. ok() is false when nothing was checked, because a gate that measured nothing has not passed -- it has not run.

**`bool ok() const`**
The ok.

Chapter 8

# The Terminal UI

How the spreadsheets got a screen, and how a terminal interface is tested.

v10 made a model's fields text, v11 made the model itself data, and v12 made the run something a caller drives. Each was built for a front end that did not exist. This is that front end -- a tab bar of module types, a grid of rows, a detail pane where editing happens, and a run view.

### The screen is a value

A renderer fills a Screen; the terminal blits it. render() is pure and handleKey() touches nothing outside the state, so a test builds a document, types a scripted key sequence, and asserts on the screen AS TEXT -- no terminal, no timing, no sleeps. That is what makes a terminal interface testable at all.

### Why the portable half lives in src/

The verification script compiles src/*.cpp, and its sanitiser leg compiles tests and src together. Putting the portable half of the interface there runs the whole of it through both compilers and both sanitisers with no change to any gate script. Only the platform layer sits outside, in tui/, and platform code cannot be tested anyway -- which is the reason it is three methods long.

### Nothing here names a module type

The tab bar is every type the registry publishes, then any the document holds that it does not. The first half is why a module added in a later version renders without this code changing. The second is why a file written by a later version shows its unknown modules instead of hiding them.

### What only looking at it found

Six layout bugs passed every assertion in the suite, because each of them passes a search of the screen text: the selected tab drawn off the edge, a file name trimmed from the wrong end, a detail pane that did not scroll, an enum hint that ran on from the value, text overflowing the frame, and a finished run reporting that its rule could not say how far along it was. The screen-as-a-value design makes rendering testable; it does not make it self-evaluating.

### Building one, rather than editing one

v13 could open, edit, save and run a model; it could not be sat down in front of cold. v14 is that: every column carries a line saying what it is for, in the SCHEMA rather than here, because the schema is what a front end reads in order to render and a second front end would need the same sentence. Enter on an enum or a reference opens a list built from referenceCandidates() -- the same function the compiler's reference pass is written in terms of, so a list cannot offer a name that is then rejected.

Escape in that list does not cancel. It drops into the text editor, because naming a block BEFORE that block exists is the ordinary way a model gets built and no list can offer that. A second Escape abandons. The list is for when you know, and it gets out of the way the instant you do not.

### The flow view reads cells, not a Model

flowOf() walks the document rather than a compiled Model, because a document with a dangling exit does not compile and that is exactly when the shape is hardest to hold in your head. It also lists a Decide's branches in the order they are tried, with its Next shown last as the else exit -- printed first, as it was at first, it read as the default path being tried first, which is the opposite of what happens.

## Attr

*enum class · Screen.hpp*

How a glyph is drawn: normal, bold, dim, reversed, or the red used for a diagnostic. The terminal turns these into escape sequences; nothing above it knows that.

**Values** Normal, Bold, Dim, Reverse, Error

## Screen

*class · Screen.hpp*

A grid of glyphs, and the decision the whole user interface rests on. A renderer fills one and the terminal blits it, so a test can render a document and assert on the text a person would read -- headless, with no timing and no sleeps. Writes outside its bounds are dropped rather than asserted: a renderer that computes one column too far has a layout bug, and a missing character is a better way to find out than a crash in front of the user.

**`Screen(int width, int height)`**
Constructor.

**`int width() const`**
The width.

**`int height() const`**
The height.

**`void clear()`**
Performs clear.

**`void put(int x, int y, char ch, Attr attr = Attr::Normal)`**
One glyph. Out of range is dropped, not an error.

**`int text(int x, int y, const std::string& s, Attr attr = Attr::Normal)`**
A run of text, returning the x it stopped at so callers can chain runs.

**`void hline(int x, int y, int length, char ch = '-', Attr attr = Attr::Normal)`**
A horizontal run of one character.

**`void box(int x, int y, int width, int height, Attr attr = Attr::Normal)`**
A frame drawn in dashes, pipes and plus signs.

**`const Glyph& at(int x, int y) const`**
The glyph at a position, or a blank one when the position is outside the screen.

**`std::string line(int y) const`**
One row as plain text with the trailing blanks trimmed. This is the test interface: an assertion says what a person would read rather than picking through an attribute grid.

**`std::string asText() const`**
Every line, newline separated. What a test compares.

## KeyKind

*enum class · Key.hpp*

What kind of keypress this is: a printable character, a control chord, or one of the named keys a terminal sends as an escape sequence.

**Values** Char, Enter, Escape, Backspace, Delete, Tab, BackTab, Up, Down, Left, Right, Home, End, PageUp, PageDown, Ctrl, Function, Mouse, Unknown

## Key

*struct · Key.hpp*

A keypress as a value. The platform layer turns a console record or an escape sequence into one of these, and everything above takes them -- which is what lets a test type a whole session without a terminal. Control keys normalise to upper case, so ^s and ^S are one key rather than two that can disagree.

**`static Key character(char c)`**
A printable key.

**`static Key control(char c)`**
A control chord, normalised to upper case.

**`static Key special(KeyKind kind)`**
One of the named keys, such as Up or Enter.

**`static Key shifted(KeyKind kind)`**
An arrow or a Home/End with Shift held, which is what extends a selection. Shift rides ALONGSIDE the kind rather than doubling it, so there is one Left in every switch rather than two that can disagree.

**`static Key function(int n)`**
A function key, carrying its number. F1 opens the key map.

**`static Key mouse(MouseButton button, int x, int y)`**
A click or a wheel turn at a screen position. Screen coordinates, so the renderer's layout decides what was hit -- the terminal layer does not know what a palette is.

**`bool isWheel() const`**
Whether wheel.

## MouseButton

*enum class · Key.hpp*

Which button, or which way the wheel turned. A click arrives as a Key like any other press, so there is one event stream and nothing has to merge two.

**Values** None, Left, Right, Middle, WheelUp, WheelDown

## TerminalSize

*struct · Terminal.hpp*

How many columns and rows the terminal has right now. Read every frame, because a person can resize a window mid-edit.

## ITerminal

*class · Terminal.hpp*

The only platform code in the project: report the size, blit a Screen, read a Key, say whether one is waiting, and name the window. It is an object rather than a handful of functions because its destructor restores the console -- a return from anywhere in the loop must not leave somebody in raw mode with no cursor. openTerminal() can refuse, with a reason: a console that was not a console used to be driven anyway, and the loop then spun forever reading nothing.

**`virtual TerminalSize size() const = 0`**
The terminal size right now.

**`virtual void present(const Screen& screen) = 0`**
Write a whole Screen. No diffing: a spreadsheet is a few thousand cells and a person types at human speed, so the optimisation would only be somewhere for a rendering bug to hide.

**`virtual Key nextKey() = 0`**
Block until a key arrives. Anything unrecognised comes back as Unknown, which the input layer ignores -- an unfamiliar escape sequence must never stop the program.

**`virtual bool keyPending() = 0`**
Is a keypress waiting? The fourth method, and the one that makes a run interruptible: while a simulation is advancing the loop must not block in nextKey(), or Escape is never seen and an unbounded model runs until the process is killed. On Win32 it also DRAINS the mouse, focus and resize records the console reports alongside keys -- otherwise the handle stays signalled and the run never advances at all, the same bug wearing the opposite face.

**`virtual void setTitle(const std::string& title) = 0`**
Names the window after the file that is open. With two models open, every window used to say the same thing, and a person could not tell which was which.

## FlowBlock

*struct · TuiState.hpp*

One flowchart block as the flow view shows it: its name, its type, and where its exits lead. Built from the DOCUMENT rather than from a compiled Model, because a document with a dangling exit does not compile -- and that is exactly when somebody needs to see the wiring. It also carries whether an exit names nothing and whether anything arrives here, neither of which is an error on its own and neither of which is said anywhere else.

## TuiState

*class · TuiState.hpp*

Everything the user interface knows: the document, where the cursor is, which mode is active, the diagnostics from the last compile, the undo stack and the run in progress. The renderer reads it and the input layer writes it, and neither touches a terminal.

**`static TuiState open(const std::string& path)`**
Reads a file. One that does not PARSE still opens, with its diagnostics shown, because the document layer preserves what it read. One that does not EXIST opens as an empty document, so pointing the editor at a new name starts a new model; the status line says which happened, because empty and unreadable must not look alike.

**`static TuiState fromText(const std::string& text, std::string path)`**
A state around text already in memory rather than a file on disk. The tests use it so the suite does not need a file for every case, and so a test that SAVES cannot accidentally depend on what a previous run left at that path.

**`View view() const`**
The view.

**`Pane focus() const`**
The focus.

**`Overlay overlay() const`**
The overlay.

**`void setView(View v)`**
Sets the view.

**`void setFocus(Pane p)`**
Sets the focus.

**`void setOverlay(Overlay o)`**
Sets the overlay.

**`const std::string& path() const`**
The path.

**`bool dirty() const`**
The dirty.

**`const std::string& status() const`**
The status.

**`void setStatus(std::string s)`**
Sets the status.

**`const TextBuffer& buffer() const`**
The text being edited. The non-const overload is how the input layer moves the caret and extends selections; anything that CHANGES the text goes through the editing members instead, so nothing can alter it without the document and the diagnostics catching up.

**`TextBuffer& buffer()`**
The text being edited. The non-const overload is how the input layer moves the caret and extends selections; anything that CHANGES the text goes through the editing members instead, so nothing can alter it without the document and the diagnostics catching up.

**`void edited()`**
Performs edited.

**`void insertText(const std::string& text)`**
Inserts text at the caret, replacing the selection, and re-reads. What a paste and the starter model both go through.

**`void typeChar(char c)`**

One character at the caret. Every keystroke re-parses and re-compiles, which for a file measured in kilobytes costs nothing worth measuring -- and which is why the compiler has to survive every half-typed prefix.

**`void newline()`**
Performs newline.

**`void backspace()`**
Performs backspace.

**`void del()`**
Performs del.

**`void undo()`**
Pops a whole-document snapshot. It leaves the document dirty on purpose -- undoing back to what is on disk is not the same as knowing you are there, and claiming otherwise loses work.

**`void redo()`**
Performs redo.

**`void cut()`**
Performs cut.

**`void copy()`**
Performs copy.

**`void paste()`**
Performs paste.

**`const std::string& clipboard() const`**
The clipboard.

**`void setClipboard(std::string s)`**
Sets the clipboard.

**`bool save()`**
Writes the document back. Records nobody edited come out exactly as they were read, comments included.

**`bool save(const std::string& toPath)`**
Writes the document back. Records nobody edited come out exactly as they were read, comments included.

**`const ModelDocument& document() const`**
The document.

**`const std::vector<Diagnostic>& diagnostics() const`**
The diagnostics.

**`const Diagnostic* diagnosticOnLine(std::size_t line) const`**
The first diagnostic on a 1-based file line, or null. An ERROR beats a warning on the same line: one marker per line, and the one worth showing is the one that stops the model running.

**`std::size_t firstErrorLine() const`**
The first error line.

**`void goToFirstError()`**
Performs go to first error.

**`const std::vector<std::string>& palette() const`**
The palette.

**`std::size_t paletteIndex() const`**
The palette index.

**`void stepPalette(long long delta)`**
Moves the selection in the module palette. Clamps rather than wraps, like every other cursor here.

**`void setPaletteIndex(std::size_t i)`**
Sets the palette index.

**`void insertSelectedModule()`**
Performs insert selected module.

**`void showModuleHelp()`**
Performs show module help.

**`void showFieldHelp()`**
Performs show field help.

**`const std::string& helpTitle() const`**
The help title.

**`const std::vector<std::string>& helpBody() const`**
The help body.

**`void clearHelp()`**
Performs clear help.

**`bool openPicker()`**
Arena's drop-down, on the `key = value` line the caret is on. An Enum offers its spellings; a Reference offers what exists, built by the same function the compiler's reference pass checks against, so a list cannot offer a name that is then rejected. Returns false, having opened nothing, when the field has no list or when nothing has been declared yet -- and says why, because on a model being built from scratch that is the ordinary case.

**`const std::vector<std::string>& choices() const`**
The choices.

**`std::size_t choice() const`**
The choice.

**`void stepPicker(long long delta)`**
Moves the cursor in the open list. Clamps rather than wraps.

**`void commitPicker()`**
Performs commit picker.

**`void cancelPicker()`**
Performs cancel picker.

**`const std::vector<std::string>& runs() const`**
The runs.

**`std::size_t runIndex() const`**
The run index.

**`void stepRun(long long delta)`**
Chooses among the [Run] records the file holds. v15 allows several; v11 through v14 reported the second as an error.

**`void setRunIndex(std::size_t i)`**
Sets the run index.

**`void addRunBlock()`**
Performs add run block.

**`void startRun()`**
Compiles and enters the run mode, or refuses with a reason. Entering a run mode with nothing running is worse than not entering it.

**`void advanceRun()`**
Does a budget of events and captures the report when the run ends.

**`void stopRun()`**
Cancels and returns to the grid.

**`const RunController* running() const`**
The running.

**`RunsFocus runsFocus() const`**
The runs focus.

**`void setRunsFocus(RunsFocus f)`**
Sets the runs focus.

**`const std::vector<std::string>& runFields() const`**
The run fields.

**`std::size_t runField() const`**
The run field.

**`void stepRunField(long long delta)`**
Moves between the Runs form's fields. Clamps rather than wraps.

**`void setRunField(std::size_t i)`**
Sets the run field.

**`bool editingRunField() const`**
The editing run field.

**`const std::string& runFieldDraft() const`**
The run field draft.

**`void beginRunFieldEdit()`**
Performs begin run field edit.

**`void typeRunField(char c)`**
One character into the field being edited. Nothing reaches the text until the edit is committed.

**`void backspaceRunField()`**
Performs backspace run field.

**`bool commitRunFieldEdit()`**
Writes the edited field into the [Run] block. A value the compiler would reject is refused with its message, and the edit stays open to be fixed.

**`void cancelRunFieldEdit()`**
Performs cancel run field edit.

**`void showRunFieldHelp()`**
Performs show run field help.

**`bool setRunValue(std::size_t run, const std::string& column, const std::string& value)`**
Sets the run value.

**`void requestDeleteRun()`**
Performs request delete run.

**`bool deleteRun(std::size_t run)`**
Removes a [Run] block and the comment directly above it, as one undo step. The last block in a file takes its separating blank lines with it.

**`void requestNewFile()`**
Performs request new file.

**`void openPrompt(PromptFor why)`**

Opens the one-line name prompt, for a new model or for naming an unnamed one.

**`PromptFor promptFor() const`**
The prompt for.

**`const std::string& promptText() const`**
The prompt text.

**`void typePrompt(char c)`**
One character into the name being typed.

**`void backspacePrompt()`**
Performs backspace prompt.

**`void commitPrompt()`**
Performs commit prompt.

**`void cancelPrompt()`**
Performs cancel prompt.

**`void answerConfirm(char c)`**
Answers the NewFile and OpenExisting questions: save first, discard, or cancel; open the existing file, or go back to the name.

**`bool unnamed() const`**
The unnamed.

**`void setUnnamed(bool on)`**
Sets the unnamed.

**`void setDefaultDirectory(std::string dir)`**
Sets the default directory.

**`void newBlankModel(const std::string& path)`**
Replaces the model with an empty one at a path, clean and with nothing to undo.

**`ConfirmFor confirmFor() const`**
The confirm for.

**`PickerFor pickerFor() const`**
The picker for.

**`const std::string& pickerTitle() const`**
The picker title.

**`const std::string& results() const`**
The results.

**`const std::string& resultsOf() const`**
The results of.

**`bool haveResults() const`**
The have results.

**`std::size_t resultsScroll() const`**
The results scroll.

**`void scrollResults(long long delta)`**
Scrolls the report. A report longer than the window is the normal case for a study with replications.

**`bool saveResults()`**
Writes the last report beside the model, as .results.txt, with a first line naming the file and WHICH RUN produced it. A report that does not say which settings made it is a report you cannot file.

**`const std::string& resultsPath() const`**
The results path.

**`std::size_t flowCursor() const`**
The flow cursor.

**`void stepFlow(long long delta)`**
Moves the marker in the flow view. Enter then opens the block it rests on, which is what makes the view a way of getting somewhere rather than a second place to look.

**`void gotoFlowBlock()`**
Performs goto flow block.

**`bool wantsQuit() const`**
The wants quit.

**`void requestQuit()`**
Performs request quit.

Chapter 9

# Editing a Model

The text editor, the palette, and why the file itself is what gets edited.

v13 put a spreadsheet grid on the screen and edited a ModelDocument. v15 edits the FILE and parses it, which is the other way round and settles three things at once: the byte-identical round trip is free rather than a guarantee to keep, a value can only be wrong in one place, and comments and ordering are ordinary text a person edits rather than data a writer has to be careful with.

### Every keystroke compiles

The buffer changes, the text is re-read, the document is re-compiled, and the diagnostics are re-hung on their lines. That is affordable at this size -- and it means every PREFIX of every model anyone types is a state the compiler sees. EXPO( is one of them, on the way to EXPO(1.0), and for four versions it aborted the program: Exponential asserted on a non-positive mean where six of the nine distributions threw a ModelError, and buildCall's catch, which exists precisely so a bad parameter becomes a diagnostic, cannot catch an assert.

### A diagnostic names a cell; an editor needs a line

lineOf() is the join. A reader diagnostic carries its line already; a compiler diagnostic carries a CellRef, and a cell knows where it was read from; a diagnostic about a field that is NOT THERE has no cell at all, so the record's headerLine is the nearest true thing. It returns 0 for 'cannot say', and the renderer draws no marker rather than guessing -- which is the same rule as a progress bar that will not pretend to know.

### One layout, two readers

layoutFor() computes the geometry once. render() draws with it and the input layer hit-tests clicks with it, so a click is turned back into what was clicked by the arithmetic that drew it.

### Runs you can change, files with names

v16's Runs tab is a list and a form. Every change in the form rewrites one line of that run's [Run] block -- tried on a copy first, so a bad value never reaches the text, and one undo step. ^N asks for a model's name, and a model opened with no file asks on its first save rather than becoming untitled.des. One Confirm overlay and one Prompt serve every question, told apart by ConfirmFor and PromptFor.

## TextBuffer

*class · TextBuffer.hpp*

The text of a .des file, with a caret, a selection and an undo stack. v15's whole design rests on this being the source of truth: the document, the diagnostics, the flow and the list of runs are all re-derived from it after every change, so what is saved is exactly what was typed and the byte-identical round trip costs nothing to keep.

**`TextBuffer()`**
Constructor.

**`explicit TextBuffer(const std::string& text)`**
Constructor.

**`void setText(const std::string& text)`**
Sets the text.

**`std::string text() const`**
The text.

**`const std::vector<std::string>& lines() const`**
The lines.

**`std::size_t lineCount() const`**
The line count.

**`const std::string& lineAt(std::size_t i) const`**
One line, or an empty string past the end. Out of range returns rather than throws: a renderer that computes one line too far has a layout bug, and a crash in front of the user is a worse way to find out.

**`Caret caret() const`**
The caret.

**`void moveTo(Caret to, bool extend = false)`**
Puts the caret at a position, clamping it into the text. With extend it keeps the selection anchor where it was, which is what Shift+click and Shift+arrow do.

**`void moveBy(long long lines, long long columns, bool extend = false)`**
Moves the caret by lines and columns. Off either end of a line steps to the next or the previous one, which is what makes Left at column 0 useful.

**`void moveToLineStart(bool extend = false)`**
Home.

**`void moveToLineEnd(bool extend = false)`**
End.

**`void moveToStart(bool extend = false)`**
To the top of the text.

**`void moveToEnd(bool extend = false)`**
To the end of the text, which is where a new record is appended.

**`bool hasSelection() const`**
Whether selection.

**`Caret selectionStart() const`**
The selection start.

**`Caret selectionEnd() const`**
The selection end.

**`void selectAll()`**
Performs select all.

**`void clearSelection()`**
Performs clear selection.

**`std::string selectedText() const`**
The selected text.

**`void insert(const std::string& text)`**
Inserts text at the caret, replacing the selection first. Text carrying newlines splits the line, so a paste behaves exactly like typing -- and a clipboard almost always carries them.

**`void newline()`**
Performs newline.

**`void backspace()`**
Performs backspace.

**`void del()`**
Performs del.

**`bool deleteSelection()`**
Removes the selection and joins what is left, returning false when there was none.

**`void insertLines(const std::vector<std::string>& block)`**
A block of whole lines, which is what the module palette inserts. It lands on a LINE BOUNDARY -- a partly-typed line is split rather than cut through, since a [Create] record dropped into the middle of an expression would leave two broken lines -- and the caret ends on the block's first field rather than on its header, so every insertion does not start with the same two keystrokes.

**`bool canUndo() const`**
The can undo.

**`bool canRedo() const`**
The can redo.

**`void undo()`**
Performs undo.

**`void redo()`**
Performs redo.

**`void checkpoint()`**

Performs checkpoint.

## Caret

*struct · TextBuffer.hpp*

A position in a TextBuffer: a line and a column, both counted from zero. A column may equal the line's length, which is the position after the last character and where typing usually happens.

## View

*enum class · TuiState.hpp*

What fills the body of the window -- the model, the flow, the runs or the results. Each is reachable from the bar across the top AND from one control key, because a bar you can only reach with a mouse is a bar half the people using it cannot reach at all.

**Values** Model, Flow, Runs, Results

## Pane

*enum class · TuiState.hpp*

Which half of the Model view has the keyboard: the module palette on the left or the text on the right.

**Values** Palette, Editor

## Overlay

*enum class · TuiState.hpp*

Something drawn OVER the view, which takes every key until it closes -- the key map, a field's help, a pick list, the confirm prompt, or a run in progress. Modal on purpose: a list of choices you could edit underneath would be answering a question about text that had moved.

**Values** None, Help, Picker, Confirm, Running, Prompt

## Layout

*struct · TuiRender.hpp*

Where everything is on screen. ONE function computes it, the renderer draws with it and the input layer hit-tests with it -- because a click has to be turned back into 'the third palette entry' by exactly the arithmetic that put it there. Two copies of that arithmetic is a bug this project has already had once, when v13's tab bar measured without the scroll markers and drew with them.

## ConfirmFor

*enum class · TuiState.hpp*

Which question the one Confirm overlay is asking -- quit, new file, delete a run, open an existing file -- so a 'd' meaning discard-and-quit can never be read as discard-and-open.

**Values** Quit, NewFile, DeleteRun, OpenExisting

## PromptFor

*enum class · TuiState.hpp*

What the one-line Prompt is for: naming a new model, or naming an unnamed one on its first save.

**Values** None, NewFile, SaveAs

## RunsFocus

*enum class · TuiState.hpp*

Which half of the Runs tab has the keyboard: the list of runs, or the chosen run's form.

**Values** List, Form

## PickerFor

*enum class · TuiState.hpp*

Which field a Picker writes into: the editor's current line, or the Runs form's selected field.

**Values** Editor, RunField

Chapter 10

# The Flowchart

The blocks entities move through, and the narrow window each one gets on the engine.

A model is a graph of blocks. Each does one job to an entity and decides where it goes next. A block reaches the engine only through NodeContext, which exposes what a block may do and nothing else: it cannot touch the future event list, the statistics or the clock.

## NodeContext

*class · Node.hpp*

The narrow facade through which a block touches the engine. A node can create, route, schedule and destroy, and cannot reach the future event list, the statistics or the clock.

**`explicit NodeContext(SimulationSystem& sim) : m_sim(sim)`**
Constructor.

**`SimTime now() const`**
The current simulation time.

**`RandomStream& rng()`**
The shared random stream.

**`Trace& trace()`**
The trace writer.

**`void route(Entity* e, INode* to)`**
Hands the entity to the next block. A null target means it leaves the system.

**`void scheduleReturn(SimTime at, Entity* e, INode* node)`**
Asks to be called back at a given time with this entity -- how a Process schedules the end of a service without seeing the event list.

**`void scheduleRenegeCheck(SimTime at, Entity* e, INode* node)`**
Schedules a patience timer.

**`void scheduleNextArrival(SimTime at, INode* source)`**
A source asking to produce again. A separate call because the event type differs.

**`void setEntityType(Entity* e, const std::string& type)`**
Retypes an entity through the engine, so the per-type counts move with it.

**`EvalContext evaluationContext(const Entity* e)`**
Builds the EvalContext an expression needs. The only route by which a block reaches the variables and the model state.

**`VariableStore& variables()`**
The global variable store.

**`Entity* createEntity()`**
Makes a new entity.

**`void registerArrival(Entity* e)`**
Marks an entity as demand the system received. Only a Create block calls it; batch representatives and duplicates are manufactured, not arrivals.

**`void destroy(Entity* e)`**
Destroys an entity.

## INode

*class · Node.hpp*

A block in the flowchart. Does one job to an entity and decides where it goes next. Everything from Create to Dispose is one of these, which is why the engine has no special case for arrivals.

**`explicit INode(std::string name) : m_name(std::move(name))`**
Constructor.

**`const std::string& name() const`**
The name this object was constructed with.

**`INode* next() const`**
The default onward block.

**`virtual void setNext(INode* n)`**
Sets the block this one routes to by default.

**`virtual void enter(NodeContext& ctx, Entity* e) = 0`**
An entity has arrived at this block. Does the block's job and routes onward -- possibly not immediately.

**`virtual void onScheduledEvent(NodeContext& ctx, Entity* e)`**
A previously scheduled return has come due, such as the end of a service or a delay.

**`virtual void onRenegeTimeout(NodeContext& , Entity* /*e*/)`**
A patience timer has expired. Timers are never cancelled, so this checks whether the entity is still waiting and ignores the event if not.

**`virtual void reset()`**
Back to the t=0 condition. Configuration survives; run state does not.

**`virtual void resetStatistics(SimTime )`**
Discards measurements collected so far and starts again from now, without touching what is held. This is warm-up removal at block level.

**`virtual std::string describe() const = 0`**
A human-readable one-line description, used in model dumps and traces.

**`virtual double loadPerVisit() const`**
Mean resource-time consumed per entity passing through, divided by capacity. Used by the stability check.

**`virtual bool loadIsKnown() const`**
False when the mean cannot be computed in advance, which is not the same as no load.

## DelayNode

*class · Nodes.hpp · inherits public INode*

Holds an entity for a duration and passes it on. No resource, no queue, no contention -- transport, cooling, walking between rooms.

**`DelayNode(std::string name, std::unique_ptr<IDistribution> duration)`**
Constructor.

**`DelayNode(std::string name, ExpressionPtr duration)`**
Constructor.

**`const IExpression& durationExpression() const`**
The expression giving the hold time.

**`IExpression& durationExpression()`**
The expression giving the hold time.

**`void enter(NodeContext& ctx, Entity* e) override`**
An entity has arrived at this block. Does the block's job and routes onward -- possibly not immediately.

**`void onScheduledEvent(NodeContext& ctx, Entity* e) override`**
A previously scheduled return has come due, such as the end of a service or a delay.

**`void reset() override`**
Back to the t=0 condition. Configuration survives; run state does not.

**`void resetStatistics(SimTime now) override`**
Discards measurements collected so far and starts again from now, without touching what is held. This is warm-up removal at block level.

**`std::string describe() const override`**
A human-readable one-line description, used in model dumps and traces.

**`const Statistics& stats() const`**
The statistics accumulator for this block.

**`int inTransit() const`**
The in transit.

## AssignTarget

*enum class · Nodes.hpp*

What an Assign writes: an entity attribute, a global variable, or the entity's type.

**Values** Attribute, Variable, EntityType

## AssignNode

*class · Nodes.hpp · inherits public INode*

Sets one or more values on the entity passing through, instantly. Fields run in order, so a later field sees what an earlier one wrote.

**`explicit AssignNode(std::string name)`**
Constructor.

**`AssignNode& set(const std::string& attribute, std::unique_ptr<IDistribution> value)`**
Adds a field to this block. Fields run in the order they were added.

**`AssignNode& set(AssignTarget target, const std::string& name, ExpressionPtr value)`**
Adds a field to this block. Fields run in the order they were added.

**`const std::vector<Rule>& rules() const`**
The fields this block writes, in order.

**`void enter(NodeContext& ctx, Entity* e) override`**
An entity has arrived at this block. Does the block's job and routes onward -- possibly not immediately.

**`void reset() override`**
Back to the t=0 condition. Configuration survives; run state does not.

**`void resetStatistics(SimTime now) override`**
Discards measurements collected so far and starts again from now, without touching what is held. This is warm-up removal at block level.

**`std::string describe() const override`**
A human-readable one-line description, used in model dumps and traces.

**`long long count() const`**
How many entities have passed through.

## DecideNode

*class · Nodes.hpp · inherits public INode*

Branches. Either by chance, from a single draw walked against cumulative probabilities, or by condition, first match wins. A block is all-chance or all-condition, never mixed.

**`DecideNode(std::string name, double probability)`**
Constructor.

**`DecideNode(std::string name, Condition condition)`**
Constructor.

**`DecideNode(std::string name, ExpressionPtr condition)`**
Constructor.

**`explicit DecideNode(std::string name, bool byChance)`**
Constructor.

**`DecideNode& addBranch(double probability, INode* target)`**
Adds a branch, in order. Order is meaningful for conditions: first match wins.

**`DecideNode& addBranch(Condition condition, INode* target)`**
Adds a branch, in order. Order is meaningful for conditions: first match wins.

**`DecideNode& addBranch(ExpressionPtr condition, INode* target)`**
Adds a branch, in order. Order is meaningful for conditions: first match wins.

**`void setTrueBranch(INode* n)`**
Sets the target of the first branch -- the two-way spelling.

**`INode* trueBranch() const`**
The true branch.

**`void enter(NodeContext& ctx, Entity* e) override`**
An entity has arrived at this block. Does the block's job and routes onward -- possibly not immediately.

**`void reset() override`**
Back to the t=0 condition. Configuration survives; run state does not.

**`void resetStatistics(SimTime now) override`**
Discards measurements collected so far and starts again from now, without touching what is held. This is warm-up removal at block level.

**`std::string describe() const override`**
A human-readable one-line description, used in model dumps and traces.

**`bool isByChance() const`**
Whether this block branches by chance rather than by condition.

**`double probability() const`**
The probability.

**`const std::vector<Branch>& branches() const`**
The branches, in evaluation order.

**`long long fellThrough() const`**
How many matched no branch.

**`long long tookTrue() const`**
How many entities took the first branch.

**`long long tookFalse() const`**
How many took any other branch or fell through.

## BatchNode

*class · Nodes.hpp · inherits public INode*

Accumulates entities and sends one representative onward carrying them. Permanent batches consume their members; temporary ones can be split again later. Can group by matching or by distinct attribute values.

**`BatchNode(std::string name, std::size_t size, bool permanent = false)`**
Constructor.

**`void enter(NodeContext& ctx, Entity* e) override`**
An entity has arrived at this block. Does the block's job and routes onward -- possibly not immediately.

**`void reset() override`**
Back to the t=0 condition. Configuration survives; run state does not.

**`void resetStatistics(SimTime now) override`**
Discards measurements collected so far and starts again from now, without touching what is held. This is warm-up removal at block level.

**`std::string describe() const override`**
A human-readable one-line description, used in model dumps and traces.

**`std::size_t size() const`**
The size.

**`bool isPermanent() const`**
Whether permanent.

**`long long batchesFormed() const`**
The batches formed.

**`std::size_t waitingForBatch() const`**
The waiting for batch.

**`std::size_t maxQueueLength() const`**
The max queue length.

**`const Statistics& queueStats() const`**
The queue stats.

**`const Statistics& stats() const`**
The statistics accumulator for this block.

**`Rule rule() const`**
The rule.

## SeparateNode

*class · Nodes.hpp · inherits public INode*

Splits a temporary batch back into its members, or duplicates an entity. Has two exits -- the original and the duplicate -- which is a distinction easy to lose when reading an Arena model.

**`void setDuplicateExit(INode* n)`**
Sets the duplicate exit.

**`INode* duplicateExit() const`**
The duplicate exit.

**`void setPercentToDuplicates(double pct)`**
Sets the percent to duplicates.

**`double percentToDuplicates() const`**
The percent to duplicates.

**`explicit SeparateNode(std::string name)`**
Constructor.

**`SeparateNode(std::string name, int copies)`**
Constructor.

**`void enter(NodeContext& ctx, Entity* e) override`**
An entity has arrived at this block. Does the block's job and routes onward -- possibly not immediately.

**`void reset() override`**
Back to the t=0 condition. Configuration survives; run state does not.

**`void resetStatistics(SimTime now) override`**
Discards measurements collected so far and starts again from now, without touching what is held. This is warm-up removal at block level.

**`std::string describe() const override`**
A human-readable one-line description, used in model dumps and traces.

**`long long processed() const`**
The processed.

## RecordNode

*class · Nodes.hpp · inherits public INode*

Tallies something as entities pass: a count, an attribute value, or time in system.

**`explicit RecordNode(std::string name)`**
Constructor.

**`RecordNode(std::string name, std::string attribute)`**
Constructor.

**`RecordNode(std::string name, What what, std::string attribute, ExpressionPtr value)`**
Constructor.

**`void setStatistic(std::string name)`**
Sets the statistic.

**`const std::string& statistic() const`**
The statistic.

**`What what() const`**
The what.

**`bool isCounter() const`**
Whether counter.

**`const IExpression* valueExpression() const`**
The value expression.

**`static std::unique_ptr<RecordNode> timeInSystem(std::string name)`**
Builds a Record block that tallies each entity's total time in the system.

**`void enter(NodeContext& ctx, Entity* e) override`**
An entity has arrived at this block. Does the block's job and routes onward -- possibly not immediately.

**`void reset() override`**
Back to the t=0 condition. Configuration survives; run state does not.

**`void resetStatistics(SimTime now) override`**
Discards measurements collected so far and starts again from now, without touching what is held. This is warm-up removal at block level.

**`std::string describe() const override`**
A human-readable one-line description, used in model dumps and traces.

**`long long count() const`**
How many entities have passed through.

**`double total() const`**
The total.

**`double average() const`**
The average.

**`double minimum() const`**
The minimum.

**`double maximum() const`**
The maximum.

## DisposeNode

*class · Nodes.hpp · inherits public INode*

The exit. Records the entity's departure and destroys it.

**`explicit DisposeNode(std::string name)`**
Constructor.

**`void enter(NodeContext& ctx, Entity* e) override`**
An entity has arrived at this block. Does the block's job and routes onward -- possibly not immediately.

**`void setNext(INode*) override`**
Sets the next.

**`void reset() override`**
Back to the t=0 condition. Configuration survives; run state does not.

**`void resetStatistics(SimTime now) override`**
Discards measurements collected so far and starts again from now, without touching what is held. This is warm-up removal at block level.

**`std::string describe() const override`**
A human-readable one-line description, used in model dumps and traces.

**`long long count() const`**
How many entities have passed through.

## Station

*class · Station.hpp · inherits public INode, public IResourceUser*

The Process block: seize a resource, hold the entity for a service time, release. Also the only block that queues, balks and reneges. Implements both INode and IResourceUser.

**`void enter(NodeContext& ctx, Entity* e) override`**
An entity has arrived at this block. Does the block's job and routes onward -- possibly not immediately.

**`void onScheduledEvent(NodeContext& ctx, Entity* e) override`**
A previously scheduled return has come due, such as the end of a service or a delay.

**`void onRenegeTimeout(NodeContext& ctx, Entity* e) override`**
A patience timer has expired. Timers are never cancelled, so this checks whether the entity is still waiting and ignores the event if not.

**`void reset() override`**
Back to the t=0 condition. Configuration survives; run state does not.

**`void resetStatistics(SimTime now) override`**
Discards measurements collected so far and starts again from now, without touching what is held. This is warm-up removal at block level.

**`std::string describe() const override`**
A human-readable one-line description, used in model dumps and traces.

**`double loadPerVisit() const override`**
Mean resource-time consumed per entity passing through, divided by capacity. Used by the stability check.

**`bool hasWaiting() const override`**
Whether anything is queued here.

**`SimTime headOfLineSince() const override`**
When the longest-waiting entity here started waiting. A shared resource compares this across blocks.

**`void startFromQueue(NodeContext& ctx) override`**
Starts service for the next queued entity, if a unit is available.

**`void setServiceFromAttribute(const std::string& attributeName)`**
Takes the service time from a named attribute the entity already carries.

**`bool usesServiceAttribute() const`**
The uses service attribute.

**`const std::string& serviceAttributeName() const`**
The service attribute name.

**`void setRandomStream(RandomStream* rng)`**
Sets the random stream.

**`void setBalking(std::size_t queueLength, INode* balkTo)`**
Refuses to join a queue already this long, sending the entity elsewhere instead.

**`void setReneging(std::unique_ptr<IDistribution> patience, INode* renegeTo)`**
Gives a waiting entity a random patience, after which it gives up.

**`void setReneging(ExpressionPtr patience, INode* renegeTo)`**
Gives a waiting entity a random patience, after which it gives up.

**`void setUnitsNeeded(int units)`**
Sets the units needed.

**`Resource& resource()`**
The resource this block seizes. May be shared with other blocks.

**`const Resource& resource() const`**
The resource this block seizes. May be shared with other blocks.

**`EntityQueue& queue()`**
The waiting line at this block.

**`const EntityQueue& queue() const`**
The waiting line at this block.

**`Statistics& stats()`**

The statistics accumulator for this block.

**`const Statistics& stats() const`**
The statistics accumulator for this block.

**`IExpression& serviceExpression()`**
The expression giving the service time.

**`const IExpression& serviceExpression() const`**
The expression giving the service time.

**`bool loadIsKnown() const override`**
False when the mean cannot be computed in advance, which is not the same as no load.

**`std::size_t stillWaiting() const`**
The still waiting.

**`long long balked() const`**
The balked.

**`long long reneged() const`**
The reneged.

**`int unitsHeld() const`**
Units of the resource this block currently holds -- not the same as the resource's total busy count when it is shared.

**`int unitsNeeded() const`**
The units needed.

**`SimTime drawService(NodeContext& ctx, const Entity& e)`**
Evaluates the service-time expression for this entity.

## CreateNode

*class · Create.hpp · inherits public INode*

An arrival source. Makes entities on its own schedule and reschedules itself. Because it is an ordinary INode, the engine has no separate arrival handler at all.

**`void enter(NodeContext& ctx, Entity* e) override`**
An entity has arrived at this block. Does the block's job and routes onward -- possibly not immediately.

**`void onScheduledEvent(NodeContext& ctx, Entity* e) override`**
A previously scheduled return has come due, such as the end of a service or a delay.

**`void reset() override`**
Back to the t=0 condition. Configuration survives; run state does not.

**`void resetStatistics(SimTime now) override`**
Discards measurements collected so far and starts again from now, without touching what is held. This is warm-up removal at block level.

**`std::string describe() const override`**
A human-readable one-line description, used in model dumps and traces.

**`const std::string& entityType() const`**
The entity type.

**`SimTime firstAt() const`**
The first at.

**`long long created() const`**
The created.

**`long long maxArrivals() const`**
The max arrivals.

**`int entitiesPerArrival() const`**
The entities per arrival.

**`void setPerArrival(ExpressionPtr e)`**
Sets the per arrival.

**`const IExpression* perArrivalExpression() const`**
The per arrival expression.

**`long long arrivals() const`**
The arrivals.

**`IExpression& interarrival()`**
The expression giving the time until the next arrival. Evaluated with NO entity, because the entity does not exist yet.

**`const IExpression& interarrival() const`**
The expression giving the time until the next arrival. Evaluated with NO entity, because the entity does not exist yet.

**`bool scheduled() const`**
The scheduled.

**`bool exhausted() const`**
Whether this source has produced its maximum number of arrivals.

### v16: what a ride-hailing model needs

Cars and riders are both moving entities. A **Match** pairs one from each lane when they agree on an attribute; a **Route** carries the car to the **Station** its trip ends at; arrivals follow an **ArrivalSchedule** through the day. A Create's Per Arrival may name a Variable, and Max Arrivals counts arrivals -- so Per Arrival 80 with Max Arrivals 1 is a fleet of 80, as in Arena. examples/models/ride_hailing.des puts all of it together.

## MatchNode

*class · Match.hpp · inherits public INode*

Arena's Match. Holds entities in 2 to 5 lanes until every lane has one -- or, by attribute, one with the same value -- then releases one from each at the same instant, each by its own lane's exit. Nothing routes to the Match itself.

**`MatchNode(std::string name, std::string attribute)`**
Constructor.

**`void addLane(MatchLaneNode* lane)`**
Registers a lane, in order. Model::matchLane is the way to call it.

**`void enter(NodeContext& ctx, Entity* e) override`**
An entity has arrived at this block. Does the block's job and routes onward -- possibly not immediately.

**`void reset() override`**
Back to the t=0 condition. Configuration survives; run state does not.

**`void resetStatistics(SimTime) override`**
Discards measurements collected so far and starts again from now, without touching what is held. This is warm-up removal at block level.

**`std::string describe() const override`**
A human-readable one-line description, used in model dumps and traces.

**`void tryMatch(NodeContext& ctx)`**
Releases every match now possible -- normally none or one. All matched entities are taken before any is routed, and an arrival that routing causes only queues: the loop already running looks again.

**`const std::string& attribute() const`**
The attribute.

**`bool byAttribute() const`**
The by attribute.

**`std::size_t laneCount() const`**
The lane count.

**`const MatchLaneNode& lane(std::size_t i) const`**
The i-th lane, in lane order.

**`long long matches() const`**
The matches.

## MatchLaneNode

*class · Match.hpp · inherits public INode*

One lane of a Match, and a block of its own: other blocks route to it by name, it keeps its own queue and statistics, and its Next is where its matched entity goes. A loop back into a lane is a population waiting for partners, not a routing loop.

**`MatchLaneNode(std::string name, MatchNode* owner)`**
Constructor.

**`void enter(NodeContext& ctx, Entity* e) override`**
An entity has arrived at this block. Does the block's job and routes onward -- possibly not immediately.

**`void reset() override`**
Back to the t=0 condition. Configuration survives; run state does not.

**`void resetStatistics(SimTime now) override`**
Discards measurements collected so far and starts again from now, without touching what is held. This is warm-up removal at block level.

**`std::string describe() const override`**
A human-readable one-line description, used in model dumps and traces.

**`const MatchNode& owner() const`**
The owner.

**`const std::deque<Waiting>& waitingList() const`**
The waiting list.

**`Entity* take(std::size_t i, SimTime now)`**
Removes the i-th waiting entity and counts its wait. Only the Match calls it.

**`std::size_t waiting() const`**
The waiting.

**`std::size_t maxWaiting() const`**
The max waiting.

**`double averageWaiting(SimTime now) const`**
The time-average number waiting since the start, or since the warm-up ended.

**`double averageWait() const`**
The average wait.

**`long long matched() const`**
The matched.

## StationNode

*class · Transfer.hpp · inherits public INode*

Arena's Station: a place entities arrive at, by a Route naming it or by an ordinary Next. Takes no time.

**`explicit StationNode(std::string name) : INode(std::move(name))`**
Constructor.

**`void enter(NodeContext& ctx, Entity* e) override`**
An entity has arrived at this block. Does the block's job and routes onward -- possibly not immediately.

**`void reset() override`**
Back to the t=0 condition. Configuration survives; run state does not.

**`void resetStatistics(SimTime) override`**
Discards measurements collected so far and starts again from now, without touching what is held. This is warm-up removal at block level.

**`std::string describe() const override`**
A human-readable one-line description, used in model dumps and traces.

**`long long arrivals() const`**
The arrivals.

## RouteNode

*class · Transfer.hpp · inherits public INode*

Arena's Route: holds an entity for its route time, then delivers it to the Station named in its Destination -- no line drawn. It has no Next, so the loop check and the visit ratios stop at a Route rather than following a teleport as if it were a wire.

**`RouteNode(std::string name, ExpressionPtr time)`**
Constructor.

**`void setNext(INode* n) override`**
Sets the next.

**`void setDestination(StationNode* s)`**
Sets the destination.

**`StationNode* destination() const`**
The destination.

**`void enter(NodeContext& ctx, Entity* e) override`**
An entity has arrived at this block. Does the block's job and routes onward -- possibly not immediately.

**`void onScheduledEvent(NodeContext& ctx, Entity* e) override`**
A previously scheduled return has come due, such as the end of a service or a delay.

**`void reset() override`**
Back to the t=0 condition. Configuration survives; run state does not.

**`void resetStatistics(SimTime now) override`**
Discards measurements collected so far and starts again from now, without touching what is held. This is warm-up removal at block level.

**`std::string describe() const override`**
A human-readable one-line description, used in model dumps and traces.

**`const IExpression& timeExpression() const`**
The route time, mutable so the engine can give it its own random stream.

**`IExpression& timeExpression()`**
The route time, mutable so the engine can give it its own random stream.

**`long long travelled() const`**
The travelled.

**`long long inTransit() const`**
The in transit.

**`double averageInTransit(SimTime now) const`**
The time-average number on this Route -- the share of the fleet that is driving rather than waiting.

## LevelIntegral

*class · LevelIntegral.hpp*

The time-average of a count that steps up and down -- entities in a Match lane, entities on a Route. Each block keeps its own and updates it when the count changes, so nothing visits it on every event.

**`void change(SimTime now, long long delta)`**
Closes the rectangle up to now at the old level, then moves the level.

**`void restart(SimTime now)`**
Warm-up: forgets the area and keeps the level, because whoever is waiting still is.

**`void reset()`**
Back to the t=0 condition. Configuration survives; run state does not.

**`long long level() const`**
The level.

**`double average(SimTime now) const`**
The time-average up to now, closing the last rectangle without changing anything.

## TimeUnit

*enum class · Schedule.hpp*

Seconds, Minutes, Hours or Days. The model has one, on its [Run]; a Schedule has its own, and the compiler converts once.

**Values** Seconds, Minutes, Hours, Days

## ScheduleSlot

*struct · Schedule.hpp*

One row of an arrival schedule, already converted to the model's base unit: a rate per base unit and how long it lasts.

## ArrivalSchedule

*class · Schedule.hpp*

Rate and duration rows, repeated after the last one. nextArrival() generates a non-stationary Poisson stream by THINNING -- candidates at the peak rate, each kept with probability rate(t) / peak -- which is exact for any shape, a rate of zero included, where drawing at the current rate overshoots the busy hour that starts a minute later.

**`explicit ArrivalSchedule(std::vector<ScheduleSlot> slots)`**
Constructor.

**`double rateAt(SimTime t) const`**
The rate at a time, repeating after the last row.

**`double peak() const`**
The peak.

**`double cycle() const`**
The cycle.

**`const std::vector<ScheduleSlot>& slots() const`**
The slots.

**`SimTime nextArrival(SimTime now, RandomStream& rng) const`**
The first arrival after now, by thinning. Exact for any schedule, including hours with no arrivals at all.

## ScheduledGapExpression

*class · Schedule.hpp · inherits public IExpression*

A scheduled Create's interarrival. Evaluating it gives the gap to the next arrival, so the Create schedules it like any other; its mean is unknown on purpose, which is what tells the stability check it cannot judge this source.

**`ScheduledGapExpression(std::string scheduleName, ArrivalSchedule schedule)`**
Constructor.

**`Value evaluate(EvalContext& ctx) const override`**
Computes this node's value in the given context.

**`void validate(const ValidationContext&, std::vector<Diagnostic>&) const override`**
Appends every problem found to the list. Does not throw and does not stop at the first.

**`std::string describe() const override`**
A human-readable one-line description, used in model dumps and traces.

**`ExpressionPtr clone() const override`**
A deep copy. Needed because a distribution may be held in two places at once.

**`void useStream(RandomStream* s) override`**
Gives every sampling site beneath this node its own random stream.

**`const ArrivalSchedule& schedule() const`**
The schedule.

Chapter 11

# The Model

What is simulated, kept separate from the machinery that simulates it.

The engine takes a Model and runs it, and knows nothing about tellers or restaurants. Model owns the blocks, the resources and the variables, and validates the flowchart before a run, catching a routing loop as an error rather than as a program that never stops.

### Nested types

**ArrivalAttribute** is a name and a distribution, stamped on every arrival. **VisitRatios** is how often an average entity reaches each block, with an exact flag that goes false when a condition-based Decide makes the split an output of the run rather than an input. **StabilityReport** is what the offered-load check could and could not judge.

## Model

*class · Model.hpp*

What is simulated, as opposed to the machinery that simulates it. Owns the blocks, the resources and the variables, wires them into a flowchart, and validates the result before a run.

**`Model& arrivals(std::unique_ptr<IDistribution> d)`**
The single-source shorthand: creates a source called Arrivals.

**`Model& arrivals(const std::string& interarrivalText)`**
The single-source shorthand: creates a source called Arrivals.

**`Model& perArrival(const std::string& createName, const std::string& text)`**
Per Arrival as an expression, worked out at each arrival -- so a fleet size can live in a Variable.

**`Model& allowOverload(bool on = true)`**
Permits a model whose queues grow without bound. Correct for a terminating run, meaningless for a steady-state study, and the report says so.

**`bool overloadAllowed() const`**
Whether overload has been permitted.

**`void setInterarrival(std::unique_ptr<IDistribution> d)`**
Sets the interarrival.

**`Model& variable(const std::string& name, double initialValue = 0.0)`**
Declares a global variable. Checks it does not collide with an attribute name.

**`VariableStore& variables()`**
The variable store.

**`const VariableStore& variables() const`**
The variable store.

**`Model& attribute(const std::string& name, std::unique_ptr<IDistribution> d)`**
Declares an attribute stamped on every arrival, drawn from a distribution.

**`void assignOnArrival(const std::string& n, std::unique_ptr<IDistribution> d)`**
Alias for attribute(): stamps a drawn value on every arriving entity.

**`const std::vector<ArrivalAttribute>& arrivalAttributes() const`**
The attributes stamped on every arrival.

**`Model& resource(const std::string& name, int capacity)`**
Declares a shared resource that several blocks may seize.

**`std::size_t resourceCount() const`**
How many shared resources the model declares. Name lookup cannot enumerate, and a snapshot has to.

**`const Resource& resourceAt(std::size_t i) const`**
One declared resource, by position.

**`Resource* resourceNamed(const std::string& name)`**
Looks a resource up by name.

**`const Resource* resourceNamed(const std::string& name) const`**
Looks a resource up by name.

**`Model& decideNWayByChance(const std::string& name)`**
Starts an N-way Decide branching by chance.

**`Model& decideNWayByCondition(const std::string& name)`**
Starts an N-way Decide branching by condition.

**`Model& branch(const std::string& decideName, double probability, const std::string& to)`**
Adds a branch to an N-way Decide, in order.

**`Model& delay(const std::string& name, std::unique_ptr<IDistribution> duration)`**
Adds a Delay block.

**`Model& delay(const std::string& name, const std::string& durationText)`**
Adds a Delay block.

**`Model& assign(const std::string& name)`**
Adds or extends an Assign block. Calling it again with the same name adds another field.

**`Model& assignEntityType(const std::string& block, const std::string& typeText)`**
Restamps the entity's type from an expression.

**`Model& decideByChance(const std::string& name, double probabilityTrue)`**
Adds a two-way Decide taking the true branch with a fixed probability.

**`Model& decideByCondition(const std::string& name, DecideNode::Condition condition)`**
Adds a two-way Decide on a predicate or expression.

**`Model& decideByCondition(const std::string& name, ExpressionPtr condition)`**
Adds a two-way Decide on a predicate or expression.

**`Model& decideWhen(const std::string& name, const std::string& conditionText)`**
Adds a two-way Decide whose condition is written as text.

**`Model& batch(const std::string& name, std::size_t size, bool permanent = false)`**
Accumulates entities into groups of a given size.

**`Model& separate(const std::string& name)`**
Splits a temporary batch back into its members.

**`Model& duplicate(const std::string& name, int copies)`**
Copies an entity a given number of times.

**`Model& record(const std::string& name)`**
Counts entities passing a point.

**`Model& recordAttribute(const std::string& name, const std::string& attributeName)`**
Tallies an attribute's value as entities pass.

**`Model& recordTimeInSystem(const std::string& name)`**
Tallies time in system as entities pass.

**`Model& dispose(const std::string& name)`**
Adds a named exit.

**`Model& stationBlock(const std::string& name)`**
A Station, named for the Routes that send entities to it.

**`Model& routeBlock(const std::string& name, const std::string& timeText)`**
A Route with its route time. Its Destination is set by routeTo(), once the Station exists.

**`Model& routeTo(const std::string& routeName, const std::string& stationName)`**
Where a Route delivers. Refuses anything that is not a Station.

**`Model& match(const std::string& name, const std::string& attribute)`**
A Match; an empty attribute means Any. Kept out of the default entry, since nothing enters a Match by its own name.

**`Model& matchLane(const std::string& matchName, const std::string& laneName)`**
Adds a lane to a Match, in lane order. The lane is a block other blocks route to by name.

**`Model& route(const std::string& from, const std::string& to)`**
Connects one block's default exit to another block.

**`Model& routeTrue(const std::string& decideName, const std::string& to)`**
Connects a Decide's true branch.

**`Model& routeDuplicate(const std::string& separateName, const std::string& to)`**
Connects a Separate's duplicate exit. Its original exit is plain route().

**`Model& entryAt(const std::string& name)`**
Names the block arrivals enter at.

**`void connect(const std::string& f, const std::string& t)`**
Alias for route().

**`void setEntry(const std::string& n)`**
Alias for entryAt().

**`INode* node(const std::string& name)`**
Looks a block up by name.

**`const INode* node(const std::string& name) const`**
Looks a block up by name.

**`INode* entry() const`**
The block arrivals enter at.

**`template <typename T> T& nodeAs(const std::string& name)`**
Looks a block up by name and type, throwing if either is wrong rather than returning a null the caller will dereference later.

**`Station* station(const std::string& name)`**
Looks a Process block up by name.

**`const Station* station(const std::string& name) const`**
Looks a Process block up by name.

**`std::size_t stationCount() const`**
How many Process blocks the model has.

**`Station& stationAt(std::size_t i)`**
The i-th Process block. Process blocks are the only ones a utilisation table means anything for.

**`const Station& stationAt(std::size_t i) const`**
The i-th Process block. Process blocks are the only ones a utilisation table means anything for.

**`std::size_t sourceCount() const`**
How many arrival sources the model has.

**`CreateNode& sourceAt(std::size_t i)`**
The i-th arrival source.

**`const CreateNode& sourceAt(std::size_t i) const`**
The i-th arrival source.

**`std::size_t nodeCount() const`**
How many blocks the model has in total.

**`INode& nodeAt(std::size_t i)`**
The i-th block, of any kind.

**`const INode& nodeAt(std::size_t i) const`**
The i-th block, of any kind.

**`StabilityReport stability() const`**
What the offered-load check could and could not verify.

**`bool arrivalRateIsKnown() const`**
False when any source's interarrival mean cannot be computed in advance, which makes every station's load unknowable too.

**`std::vector<Diagnostic> checkExpressions() const`**
Validates every expression in the model against the declared names, returning diagnostics rather than throwing.

**`std::vector<Diagnostic> checkStructure() const`**
The check structure.

**`VisitRatios visitRatios() const`**
How many times an average entity passes through each block.

**`double offeredLoad(const Station& s) const`**
The offered load at one Process, summed over every arrival stream and weighted by visits.

**`void wireSources()`**
Connects the shorthand source to the entry block, so the order a model is described in does not matter.

**`void reset()`**
Back to the t=0 condition. Configuration survives; run state does not.

**`void validate() const`**
Checks the whole model and throws on the first problem. Catches routing loops before they become a hang.

**`std::string describe() const`**
The whole model as text, block by block.

Chapter 12

# Running a Simulation

The engine, the stopping rules, and the trace that makes a run checkable by hand.

SimulationSystem owns the clock, the event list, the entities, the random streams and the model. It also implements IModelState, which is how an expression reads the running system without the expression layer depending on the model.

## SimulationSystem

*class · SimulationSystem.hpp · inherits public IModelState*

The engine. Owns the clock, the future event list, the entities, the random streams and the model, and runs the loop that drives them. Also implements IModelState so expressions can read the running system.

**`explicit SimulationSystem(unsigned seed = 12345u)`**
Constructor.

**`double queueLength(const std::string& blockName) const override`**
How many entities wait at a named block. Part of IModelState, so expressions can read it. Throws on an unknown name rather than reading as an empty queue.

**`double resourceBusy(const std::string& name) const override`**
Units of a named resource currently seized. A Process with a private resource is addressable by the block's name.

**`double resourceCapacity(const std::string& name) const override`**
A named resource's capacity.

**`double numberInSystem() const override`**
How many entities are anywhere in the system.

**`SimTime now() const override`**
The current simulation time.

**`double variableAverage(const std::string& name) const`**
The time-weighted average of a global variable.

**`void reportStability(std::ostream& os) const`**
Prints the blocks the offered-load check could not verify. Prints nothing when everything was checked.

**`Model& model()`**
The model being simulated.

**`const Model& model() const`**
The model being simulated.

**`void setTermination(std::unique_ptr<ITerminationRule> rule)`**
Sets an arbitrary stopping rule.

**`void setWarmUp(SimTime t)`**
Discards statistics collected before this time.

**`void setObservationInterval(SimTime dt)`**
Samples number-in-system on a fixed grid, for Welch plots.

**`SimulationSystem& useSeparateStreams(bool on = true)`**
Gives each named role its own random stream, which is what makes two model variants comparable.

**`SimulationSystem& useAntithetic(bool on = true)`**
Turns on antithetic variates.

**`bool isAntithetic() const`**
Whether antithetic.

**`const std::vector<double>& observations() const`**
The observations.

**`SimTime measuredTime() const`**
The measured time.

**`SimTime warmUpEnd() const`**
The warm up end.

**`bool enableTrace(const std::string& path, TraceLevel level, bool markdown = true)`**
Starts writing an event log to a file.

**`const Clock& clock() const`**
The simulation clock.

**`const Statistics& statistics() const`**
The statistics.

**`const SystemState& state() const`**
The state.

**`const FutureEventList& fel() const`**
The fel.

**`const RandomStream& randomStream() const`**
The random stream.

**`const ITerminationRule* termination() const`**
The stopping rule currently set, so a caller can ask it how far through the run is.

**`std::size_t liveEntityCount() const`**
The live entity count.

**`Entity* createEntity()`**
Makes an entity and registers it in the entity table. Called through NodeContext, never directly by a block.

**`void initialise()`**
Validates the model, resets everything holding run state, and schedules the first arrival. Must be called before run().

**`void run()`**
The event loop: pop the earliest event, close the integrals for the interval that just ended, move the clock, then dispatch. That order is not negotiable.

**`bool canStep() const`**
Whether another event can be processed: one is pending, no stopping rule is met, and no EndSimulation event has fired. Deliberately does not test whether initialise() has been called -- that is a precondition the callers assert, and folding it in would turn a forgotten initialise() into a run that silently does nothing.

**`bool stepOnce()`**
Processes the imminent event and reports whether it did. run() is a loop over this and nothing else -- the same loop, with the caller holding the handle.

**`void report(std::ostream& os) const`**
Prints the run's results.

**`void report() const`**
Prints the run's results.

**`void reportArenaStyle(std::ostream& os) const`**
Prints the same numbers in Arena's report layout.

**`void reportArenaStyle() const`**
Prints the same numbers in Arena's report layout.

**`SimulationSystem& execute()`**
initialise() and run() in one call, for the common case.

**`RunResults results() const`**
The run's numbers, derived once.

**`std::vector<NamedStatistic> namedStatistics() const`**
The named statistics.

**`const std::map<std::string, TypeStats>& byType() const`**
Per-entity-type counts and times.

**`SimulationSystem& stopWhen(std::unique_ptr<ITerminationRule> rule)`**
Sets an arbitrary stopping rule, returning *this so it chains.

**`SimulationSystem& stopAt(SimTime t)`**
Stops the run at a given time.

**`SimulationSystem& stopAfter(int entities)`**
Stops after a given number of entities have left.

**`SimulationSystem& warmUpFor(SimTime t)`**
Discards statistics collected before this time, returning *this so it chains.

## NamedStatistic

*struct · SimulationSystem.hpp*

One number a model asked to have reported -- a Record's counter or tally, a Match lane's queue, a Route's traffic -- with the label the report prints. A flat list, so a study averages it across replications without knowing what kind of block it came from.

## StationResults

*struct · SimulationSystem.hpp*

One block's numbers after a run: served, utilisation, average wait, queue length.

## RunResults

*struct · SimulationSystem.hpp*

The whole run's numbers, derived once so that printing is only a view of them.

**`const StationResults& station(const std::string& name) const`**
One block's results, by name.

## ITerminationRule

*class · TerminationRule.hpp*

The interface a stopping condition implements. Asked after every event whether the run should end.

**`virtual bool isMet(const SimulationSystem& sim) const = 0`**
Whether this stopping condition now holds.

**`virtual std::string describe() const = 0`**
A human-readable one-line description, used in model dumps and traces.

**`virtual std::optional<double> progress(const SimulationSystem&) const`**
How far through this rule the run is, or NOTHING when it cannot tell. Nothing does not mean zero: a drained-system rule has no knowable fraction, and a progress bar that reads 0% throughout and then jumps to 100% is a lie the caller cannot detect.

**`virtual std::optional<SimTime> stopTime() const`**
The stop time.

## TimeLimit

*class · TerminationRule.hpp · inherits public ITerminationRule*

Stop when the clock reaches a given time.

**`explicit TimeLimit(SimTime maxTime)`**
Constructor.

**`bool isMet(const SimulationSystem& sim) const override`**
Whether this stopping condition now holds.

**`std::string describe() const override`**
A human-readable one-line description, used in model dumps and traces.

**`std::optional<double> progress(const SimulationSystem& sim) const override`**
The clock over the limit, clamped to 1. A time limit always knows how far through it is.

**`std::optional<SimTime> stopTime() const override`**
Its own limit. Known before the run starts, so the engine puts it on the event list and the clock stops exactly on it rather than on the first event after it.

**`SimTime maxTime() const`**
The max time.

## EntityLimit

*class · TerminationRule.hpp · inherits public ITerminationRule*

Stop after a given number of entities have left the system.

**`explicit EntityLimit(int maxEntities)`**
Constructor.

**`bool isMet(const SimulationSystem& sim) const override`**
Whether this stopping condition now holds.

**`std::string describe() const override`**
A human-readable one-line description, used in model dumps and traces.

**`std::optional<double> progress(const SimulationSystem& sim) const override`**
Entities served over the limit, clamped to 1.

## DrainedRule

*class · TerminationRule.hpp · inherits public ITerminationRule*

Stop when nothing is left in the system. Fires at the first moment the system is empty, which may be a transient gap rather than the end of the work.

**`bool isMet(const SimulationSystem& sim) const override`**
Whether this stopping condition now holds.

**`std::string describe() const override`**
A human-readable one-line description, used in model dumps and traces.

## AnyOf

*class · TerminationRule.hpp · inherits public ITerminationRule*

Stops when any composed rule says stop.

**`AnyOf& add(std::unique_ptr<ITerminationRule> rule)`**
Adds a rule; the composite stops when any of them does.

**`bool isMet(const SimulationSystem& sim) const override`**
Whether this stopping condition now holds.

**`std::string describe() const override`**
A human-readable one-line description, used in model dumps and traces.

**`std::optional<double> progress(const SimulationSystem& sim) const override`**
The LARGEST fraction any child knows, because the run ends when the FIRST rule is met and the most advanced child is the honest estimate. Nothing when no child can tell -- not zero.

**`std::optional<SimTime> stopTime() const override`**
The earliest deadline any child knows -- the composite is met when its FIRST child is, so the first knowable time is the one the clock must stop on. A child that cannot say may still end the run sooner.

**`bool empty() const`**
Whether any rules have been added. An AnyOf with none never stops the run.

## TraceLevel

*enum class · Trace.hpp*

How much detail the trace records, from off to every event.

**Values** Off, Events, Detailed

## Trace

*class · Trace.hpp*

Writes an event-by-event log, optionally as a markdown table. Traces diff, which is what makes a behaviour change show up as a diff rather than as a slightly-off average.

**`~Trace()`**
Destructor.

**`bool open(const std::string& path, TraceLevel level, bool markdown = true)`**
Starts writing a trace to a file.

**`void close()`**
Closes the trace file. Called by the destructor, so this is only for closing early.

**`bool isOn() const`**
Whether tracing is enabled. Callers check this before building a message.

**`TraceLevel level() const`**
The level.

**`long long lineCount() const`**
How many lines have been written, so a test can assert a trace was produced.

**`void note(const std::string& text)`**
Writes a free-text line into the trace, outside the event table.

Chapter 13

# Experiments

Why one run is one sample, and what to do about it.

A single run is one observation of a random variable. Quoting it to four decimals implies a precision that does not exist. Experiment runs the model many times with independent seeds and reports an interval.

**Nested types**

**Estimate** is a point estimate with its confidence interval. **Comparison** is two configurations and the interval on their difference, which is what actually answers whether a change did anything.

## ReplicationResult

*struct · Experiment.hpp*

One replication's summary figures, so a set of runs can be treated as a sample.

## Summary

*class · Experiment.hpp*

The statistics of a sample of replications: mean, standard deviation, standard error and a 95% half-width from Student's t.

**`static double mean(const std::vector<double>& xs)`**
The theoretical mean, used by the stability check before the run.

**`static double stdDev(const std::vector<double>& xs)`**
Sample standard deviation, with the n-1 denominator.

**`static double standardError(const std::vector<double>& xs)`**
Standard deviation divided by the square root of the sample size.

**`static double halfWidth95(const std::vector<double>& xs)`**
Half the width of a 95% confidence interval, from Student's t.

**`static double tCritical95(int degreesOfFreedom)`**
The two-sided 95% critical value of Student's t.

## Experiment

*class · Experiment.hpp*

Runs a model many times with independent seeds and reports an interval rather than a number. Also implements Welch's method for finding the warm-up period.

**`Experiment(std::string name, Builder build)`**
Constructor.

**`Experiment& replications(int n)`**
How many independent runs to make.

**`Experiment& baseSeed(unsigned s)`**
The seed the per-replication seeds are derived from.

**`Experiment& warmUp(SimTime t)`**
The warm-up period applied to every replication.

**`Experiment& observeEvery(SimTime dt)`**
The observation grid used for Welch's method.

**`Experiment& separateStreams(bool on = true)`**
Turns on per-role random streams for every replication.

**`Experiment& antitheticPairs(bool on = true)`**
Pairs each replication with its antithetic twin.

**`void run()`**
Runs every replication and collects the results.

**`const std::vector<ReplicationResult>& results() const`**
The results.

**`std::vector<double> column(double ReplicationResult::* field) const`**
Pulls one field out of every replication result as a plain vector, so any of them can be summarised the same way.

**`std::vector<double> waits() const`**
The waits.

**`std::vector<double> timesInSystem() const`**
The times in system.

**`std::vector<double> queueLengths() const`**
The queue lengths.

**`std::vector<double> utilisations() const`**
The utilisations.

**`static Estimate estimate(const std::vector<double>& xs)`**
A point estimate with its interval, from a sample.

**`std::vector<double> welchAverages(int window) const`**
The Welch moving average across replications, for finding the transient.

**`SimTime suggestWarmUp(int window = 0, double maxFraction = 0.5) const`**
A warm-up length suggested by MSER. A starting point, not an answer.

**`bool writeWelchSeries(const std::string& path, int window) const`**
Writes the Welch series to a CSV so it can be plotted.

**`void report() const`**
Prints each quantity as a mean with a 95% confidence interval.

Chapter 14

# Call Flows

What calls what, in order, when a model is built and run.

These traces follow examples/16_expressions.cpp, which exercises every part of the engine: text expressions, a condition reading a queue, a global variable, and a Decide by chance.

### 1 - building and starting

**`main()`**
constructs SimulationSystem(seed)

*then describes the model*

**`Model::variable / attribute / arrivals`**
declares globals and the arrival stream

*each returns *this*

**`Model::station(name, cap, rule, text)`**
parses the service expression via expr()

*throws on a bad one*

**`Model::route / routeTrue / entryAt`**
wires the flowchart

**`SimulationSystem::initialise()`**
the gate every run passes through

*Model building is ordinary method calls and nothing is simulated yet. A text field is parsed here, so a typo is caught while the model is being described rather than part way through a run.*

### 2 - initialise()

**`Model::wireSources()`**
connects the shorthand source to the entry

**`Model::validate()`**
checks the model before anything runs

*throws ModelError*

**`Model::checkExpressions()`**
every expression against declared names

*collects diagnostics*

**`Model::stability()`**
offered load per block where knowable

*names what it cannot judge*

**`reset() on everything`**
clock, stats, state, rng, FEL, entities, model

*configuration survives*

**`assignStreams()`**
a named substream per role

*if separate streams are on*

**`FutureEventList::schedule`**
the first arrival, at t=0

*Order matters here: the model is validated BEFORE run state is reset, so a broken model fails without having touched anything.*

### 3 - the run loop

**`FutureEventList::popImminent()`**
the earliest notice

**`updateAllIntegrals(notice.time())`**
closes the interval that just ENDED

*uses the OLD state*

**`Clock::advanceTo(notice.time())`**
time JUMPS; nothing happens between events

**`dispatch on notice.type()`**
Arrival, Departure, Renege, WarmUpEnd, Observe

**`ITerminationRule::isMet()`**
asked after every event

*This order is not negotiable. Closing the integrals after the clock moved would credit the new state with time it never held, and every time-weighted average would be wrong with nothing to show for it.*

### 4 - an arrival

**`CreateNode::onScheduledEvent()`**
the source's turn to produce

**`ctx.scheduleNextArrival(...)`**
reschedules ITSELF first

*before routing*

**`ctx.createEntity()`**
id and creation time assigned here

**`ctx.registerArrival(e)`**
counts it as demand, stamps attributes

**`ctx.route(e, next)`**
hands it to the first block

*synchronous*

**`Station::enter(ctx, e)`**
seize, queue, or balk

*The source reschedules before it routes. Routing is synchronous and may travel a long way through the flowchart; scheduling afterwards would make the arrival stream depend on what happens downstream.*

### 5 - service, and a freed unit



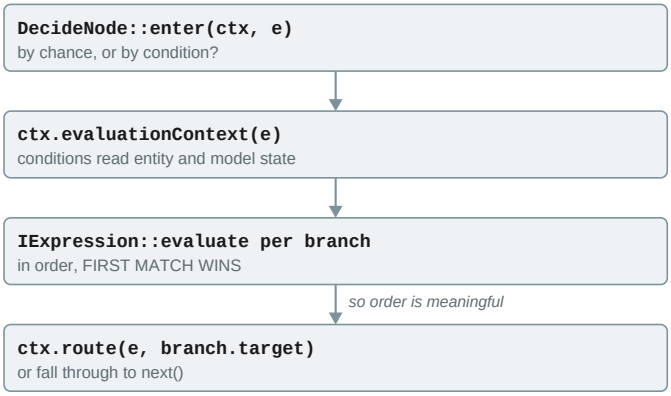

*The freed unit is offered through the resource rather than handed back to this block's own queue, because with a shared resource somebody at another block may have been waiting longer.*

### 6 - evaluating an expression




*The expression layer declares IModelState and SimulationSystem implements it, so the dependency points downward and the expression layer never includes Model.hpp.*

### 7 - a Decide by condition




*A by-chance Decide takes ONE draw and walks it against the cumulative probabilities. Drawing once per branch would burn several random numbers and would not give the branch probabilities you asked for.*

### 8 - des check model.des




*Blocks are created before their exits are wired for the same reason wireSources() exists: the order a model is described in must not matter, and route() refuses a target that does not exist yet. Every pass runs, so one command reports every bad cell rather than the first.*

Chapter 15

# Free Functions

The factories and helpers that are not members of anything.

### Distribution factories, from Build.hpp

Thin wrappers so a model reads as a description rather than as memory management.

| Function | Builds |
|---|---|
| exponential(mean) | Exponential. Takes a MEAN, not a rate. |
| constant(v) | Constant. Consumes nothing from the stream. |
| uniform(a, b) | Uniform on a closed interval. |
| triangular(a, mode, b) | Triangular. |
| normal(mean, sd, truncate) | Normal, truncated at zero by default. |
| lognormal(logMean, logSd) | Lognormal from log-space parameters. |
| lognormalFrom(mean, sd) | Lognormal from observed mean and sd. |
| weibull(scale, shape) | Weibull. |
| erlang(totalMean, phases) | Erlang with the mean you want. |
| discrete(values, probs) | Discrete, individual probabilities. |
| empirical(observations) | Empirical, sampled from data. |
| poisson(mean) | Poisson counts. |
| fixedTimes(values, repeat) | Deterministic. The basis of hand checks. |

### Termination rules, from Build.hpp

| Function | Stops when |
|---|---|
| timeLimit(t) | the clock reaches t. |
| entityLimit(n) | n entities have left. |
| whenDrained() | the system is empty, which may be a transient gap. |
| anyOf(a, b, ...) | any composed rule fires. |

### The platform boundary, from Terminal.hpp

The only functions that differ by operating system. Everything above them is a value a test can inspect; these are the part that cannot be.

| Function | Does |
|---|---|
| openTerminal(whyNot) | The console in raw mode -- or null with the reason, when input or output is redirected or the console cannot draw. |
| defaultModelPath() | Where a model goes when the program starts with no file: untitled.des in a DES Models folder inside the real Documents folder, asked of Windows so that one redirected into OneDrive is found. |
| launchedOnOwnConsole() | Whether this program is alone on its console -- double-clicked -- so a message printed on the way out needs the window held open to be read. |

### Reading a document

| Function | Does |
|---|---|
| referenceCandidates(doc, type) | Every name a Reference column may hold. The pick list and the reference pass are this one function, so a list cannot offer a name that is then rejected. |
| lineOf(doc, diagnostic) | Which line of the file a diagnostic is about, or 0 when it cannot say. A diagnostic names a cell; an editor needs a line. |
| runNames(doc) | The Name of each [Run] record, with a stand-in for one that has none. |
| readRunSetup(doc, out, which) | One [Run] record by position; past the end it reads as the defaults rather than as an error. |
| flowOf(doc) | The flowchart as the Flow view shows it, read from the cells so it still works on a model that does not compile. |
| templateFor(schema) | The lines a module inserts: its header, then every field, blank. |
| blockTypeAt / keyOfLine | Which record a line of text is inside, and the key of a key = value line. |
| starterModelText() | A working single-server model, as text, with its comments. |
| resolveModelName(typed, dir) | What a typed model name means: a bare name in the models folder, a path as given, .des added when there is no extension; blank comes back empty. |

### Time and reporting

| Function | Does |
|---|---|
| timeUnitFrom(text, out) | Seconds, Minutes, Hours or Days, from the spelling a document uses. |
| unitsPer(from, to) | How many of one unit make another: unitsPer(Hours, Minutes) is 60. |
| reportNamedStatistics(os, perRep) | Every named statistic, mean and 95% half-width across replications; nothing when there are none. |

### The expression layer

| Function | Does |
|---|---|
| tokenise(text) | Text to tokens with spans. Never throws. |
| parseExpression(text) | Tokens to a tree plus diagnostics. Never throws. |
| expr(text) | The same, but throws. For C++ callers. |
| parseExpression(text, arrays) | The same, knowing which names are array Variables, so T(1, 2) reads a cell. expr has the same overload. |
| isBuiltinFunction(name) | Whether a name is in the call table. |
| isNumber / isText | Which half of a Value is present. |
| asNumber / asText | Reads a Value, throwing rather than coercing. |
| truthy(v) | Non-zero is true; text throws. |
| formatValue(v) | A Value as text, whole numbers without a point. |
| hasErrors(diags) | Whether any diagnostic is an error. |
| formatDiagnostics(diags) | Diagnostics as text, one per line. |
| describe(TokenKind) | Names a token kind, for messages. |
| spelling(op) | An operator's symbol. |

### Grammar functions

Usable inside any expression field. Distributions: EXPO, CONS, UNIF, TRIA, NORM, LOGN, WEIB, ERLA, POIS, DISC. Array Variables: Name(i), Name(i, j). Model state: NQ(block or Match lane), NR(name), MR(name), WIP(), and the constant TNOW. Maths: MIN, MAX, ABS, ROUND, TRUNC, SQRT, LN, EXP, MOD.

**Three traps.** EXPO is the exponential distribution while EXP is e to the x, which is Arena's collision kept deliberately. DISC takes CUMULATIVE probabilities as Arena writes them, while the engine's own discrete() takes individual ones. LOGN takes the mean and standard deviation of the variable, not of its logarithm.